**UNIVERSIDAD DE GUADALAJARA**

**CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS (CUCEI)**

División de Tecnologías para la Integración Ciber-Humana

**ACTIVIDAD INTEGRADORA — REPORTE Y MANUAL TÉCNICO**

# SISTEMA DE GESTIÓN DE RECETARIO DIGITAL CON INTERFAZ GRÁFICA (wxWidgets)

Aplicación de los 4 Pilares de la Programación Orientada a Objetos, Relaciones UML, Contenedores Dinámicos (`std::vector`) y Persistencia en Disco



| | |
|---|---|
| **Nombre del Estudiante:** | Diego Ruelas Buenrostro |
| **Código de Estudiante:** | 222835378 |
| **Carrera:** | Ingeniería en Computación (ICOM) |
| **Materia:** | Programación Orientada a Objetos |
| **Clave / Sección / NRC:** | I5289 — Sección D01 — NRC 195115 |
| **Profesor:** | Mtro. Luis Angel Morales López |
| **Entorno y Herramientas:** | C++17, Biblioteca Estándar (`std::vector`, `std::sort`), Framework Gráfico **wxWidgets 3.3** |



Guadalajara, Jalisco, México — Ciclo Escolar 2026

# 1. Descripción del Problema

En la gestión gastronómica doméstica y profesional, el manejo de recetas culinarias suele realizarse de forma dispersa en notas físicas o archivos de texto sin estructura, lo que dificulta consultar rápidamente los ingredientes exactos, filtrar platillos por categoría (*Desayuno*, *Comida*, *Cena*, *Navideño*), ordenar recetas por tiempo de preparación o identificar al chef autor responsable de cada creación.

Tomando como punto de partida el prototipo preliminar de consola desarrollado previamente en la asignatura de Estructuras de Datos, el presente proyecto aborda la reingeniería y evolución completa del sistema hacia el paradigma de la **Programación Orientada a Objetos (POO)** en C++17 con una **Interfaz Gráfica de Usuario (GUI)** nativa construida con **wxWidgets**.

Para lograr un diseño limpio, mantenible y alineado con los principios de POO vistos en clase, se realizaron tres transformaciones fundamentales:

1. **Sustitución de arreglos estáticos por contenedores de la STL (`std::vector`):** Se reemplazó la plantilla manual de lista estática por el contenedor dinámico `std::vector<Recipe>` para el catálogo general, `std::vector<Ingredient>` dentro de cada receta y `std::vector<Chef>` para el registro de autores, aprovechando métodos estándar como `push_back()`, `erase()`, `clear()` y `std::sort()`.
2. **Modelado completo de los 4 Pilares de la POO y Relaciones entre Clases:** Se conservaron y refinaron las clases base del dominio (`Name`, `Ingredient` y `Recipe`) y se incorporó una jerarquía de clases para los autores culinarios conformada por la interfaz abstracta `IMostrable`, la clase base `Persona` y la clase derivada `Chef`, demostrando de manera práctica *Abstracción*, *Encapsulamiento*, *Herencia* y *Polimorfismo*.
3. **Migración de consola a Interfaz Gráfica con wxWidgets:** Se sustituyeron los menús textuales (`cout`/`cin`) por ventanas gráficas interactivas mediante herencia de clases del framework (`RecipeUI : public wxFrame` e `IngredientUI : public wxDialog`), ofreciendo tablas interactivas, filtros en tiempo real, formularios de edición y diálogos modales.

# 2. Análisis de Requerimientos del Sistema

## 2.1 Requerimientos Funcionales

- **RF-01. Gestión de Recetas (CRUD completo):** El sistema debe permitir crear nuevas recetas validando que no existan duplicados por nombre, consultar su ficha completa, modificar cualquiera de sus atributos (nombre, autor, categoría, tiempo de preparación y procedimiento) y eliminar una receta seleccionada o vaciar todo el recetario.
- **RF-02. Gestión de Ingredientes por Receta:** Cada objeto `Recipe` debe administrar internamente un `std::vector<Ingredient>`. A través del diálogo gráfico `IngredientUI`, el usuario debe poder agregar ingredientes (manteniendo el orden alfabético automáticamente), modificar su

cantidad o unidad de medida, eliminar un ingrediente individual o borrar todos los ingredientes de la receta.

- **RF-03. Búsqueda, Filtrado y Ordenamiento:** La interfaz gráfica debe permitir buscar recetas por coincidencia de nombre, filtrar por categoría culinaria (*Todas*, *Desayuno*, *Comida*, *Cena*, *Navideño*) y ordenar la colección dinámica ascendentemente tanto por nombre como por tiempo de preparación.
- **RF-04. Gestión de Chefs Autores y Demostración Polimórfica:** El sistema debe contar con un módulo dedicado para registrar objetos `Chef` (con nombre, apellido, edad, especialidad culinaria y años de experiencia), asignarlos con un clic como autores de la receta activa y consultar su ficha técnica invocando despacho dinámico polimórfico a través de referencias a la clase base `Persona` y la interfaz `IMostrable`.
- **RF-05. Persistencia en Disco:** El sistema debe serializar y deserializar el catálogo completo de recetas junto con sus listas anidadas de ingredientes hacia y desde el archivo `recetas.txt` utilizando delimitadores estructurados (`|`, `;` y `,`).

## 2.2 Requerimientos No Funcionales

- **RNF-01. Interfaz Gráfica de Usuario (GUI):** Toda la interacción con el usuario se realiza mediante controles visuales nativos de **wxWidgets** (`wxNotebook`, `wxListCtrl`, `wxChoice`, `wxSpinCtrl`, `wxTextCtrl`, `wxMessageDialog`), sin depender de la terminal.
- **RNF-02. Encapsulamiento y Modularidad:** Cada clase se divide en su archivo de cabecera (`.hpp`) en el directorio `include/` y su archivo de implementación (`.cpp`) en `src/`, protegiendo sus datos miembro como `private` o `protected`.
- **RNF-03. Uso de Contenedores Estándar C++17:** El manejo de colecciones en memoria emplea exclusivamente `std::vector` y algoritmos de `<algorithm>`, garantizando seguridad de memoria y facilidad de mantenimiento.

## 3. Diseño del Sistema y Aplicación de la POO

### 3.1 Aplicación de los 4 Pilares de la Programación Orientada a Objetos

El diseño del sistema aplica de forma explícita los cuatro pilares fundamentales de la Programación Orientada a Objetos requeridos en la Actividad Integradora:

| Pilar de la POO | Clases y Elementos Involucrados | Explicación de su Implementación en el Proyecto |
|---|---|---|
| **1. Abstracción** | `Name`, `Ingredient`, `Recipe`, `Persona`, `Chef`, `IMostrable` | Se identificaron las entidades esenciales del dominio gastronómico extrayendo únicamente sus atributos y comportamientos relevantes. Por ejemplo, `Ingredient` abstrae un insumo mediante `name`, `quantity` y `unit`; mientras que `IMostrable` abstrae el contrato de cualquier entidad capaz de generar una ficha descriptiva mediante el método virtual puro `obtenerFicha() = 0`. |
| **2. Encapsulamiento** | Atributos `private` en `Name`, `Ingredient`, `Recipe`, `Chef` y `protected` en `Persona` | Todos los atributos de estado están ocultos al exterior para impedir modificaciones directas inconsistentes. El acceso y mutación se realiza exclusivamente mediante métodos públicos *getters* y *setters* con validación (por ejemplo, en `Persona::setEdad(int)` y `Chef::setAniosExperiencia(int)` se valida que los valores no sean negativos antes de asignarlos). |
| **3. Herencia** | • `Persona : public IMostrable`<br>• `Chef : public Persona`<br>• `RecipeUI : public wxFrame`<br>• `IngredientUI : public wxDialog` | La clase derivada `Chef` hereda de la clase base `Persona` los atributos protegidos `nombre` (de tipo `Name`) y `edad`, reutilizando su lógica y especializándola con los atributos propios `especialidad` y `aniosExperiencia`. Adicionalmente, las clases gráficas heredan de `wxFrame` y `wxDialog` para adquirir el comportamiento de ventanas nativas. |
| **4. Polimorfismo** | • Métodos virtuales `obtenerRol()` y `obtenerFicha()` con `override` | Se implementa **polimorfismo de subtipado (enlace dinámico)**: al pasar un |

| Pilar de la POO | Clases y Elementos Involucrados | Explicación de su Implementación en el Proyecto |
|---|---|---|
| | • Función `consultarAutorPolimorfico(const Persona&)`<br>• Sobrecarga de constructores y operadores | objeto `Chef` a una función que recibe una referencia a la clase base `const Persona&` o un puntero `const IMostrable*`, C++ resuelve en tiempo de ejecución la versión sobrescrita (`override`) de `Chef::obtenerFicha()` y `Chef::obtenerRol()`. También se aplica **polimorfismo paramétrico/sobrecarga** en los constructores parametrizados y operadores relacionales (==, <). |

### 3.2 Diagrama de Clases UML

En la **Figura 1** se presenta el Diagrama de Clases UML completo del sistema, detallando los atributos, métodos, especificadores de visibilidad (+ público, – privado, # protegido), multiplicidades y los distintos tipos de relaciones entre clases (*Realización de Interfaz*, *Herencia / Generalización*, *Composición*, *Agregación* y *Asociación / Dependencia*).

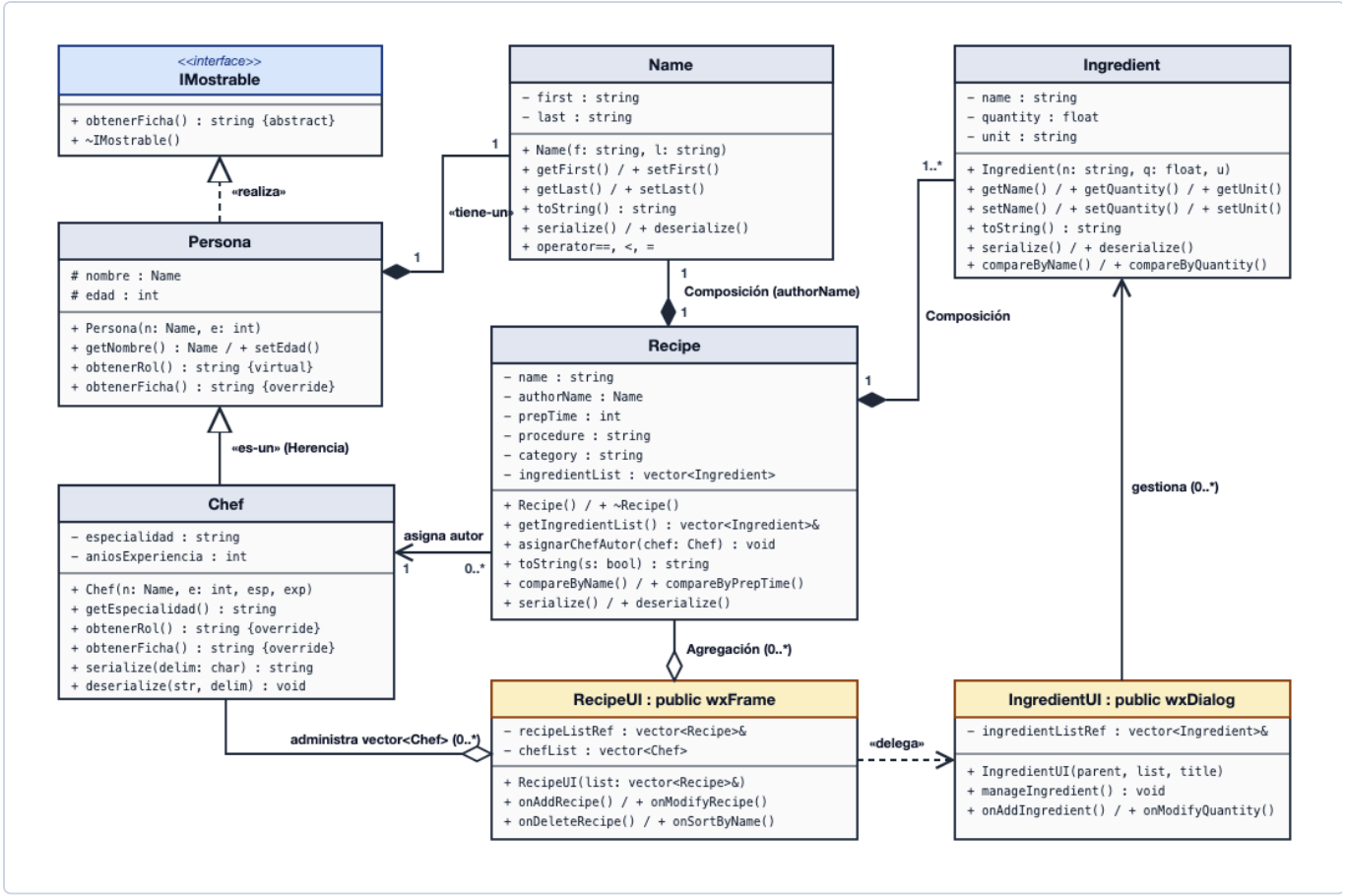



*Figura 1. Diagrama de Clases UML del Sistema de Recetario Digital en C++ con wxWidgets.*

## 3.3 Justificación de las Relaciones entre Clases

| Relación UML | Clases Participantes y Multiplicidad | Justificación Técnica en el Dominio |
|---|---|---|
| **Realización de Interfaz** («realiza») | `Persona` ——▷ `IMostrable` | `IMostrable` es una clase abstracta pura (interfaz en C++) que declara `virtual string obtenerFicha() const = 0;`. `Persona` implementa dicho contrato para garantizar que cualquier persona o subclase pueda representarse textualmente en la interfaz gráfica. |
| **Herencia / Generalización** («es-un») | `Chef` ——▷ `Persona` | Un `Chef` *es una* `Persona` especializada en gastronomía. Hereda `nombre` y `edad` de `Persona`, añade `especialidad` y `aniosExperiencia`, y redefine (`override`) los métodos virtuales `obtenerRol()` y `obtenerFicha()`. |

| Relación UML | Clases Participantes y Multiplicidad | Justificación Técnica en el Dominio |
|---|---|---|
| **Composición Fuerte** (Rombo negro lleno ◆) | • `Recipe` (1) ◆— (1..*) `Ingredient`<br>• `Recipe` (1) ◆— (1) `Name`<br>• `Persona` (1) ◆— (1) `Name` | Una receta posee por valor su propio contenedor `std::vector<Ingredient> ingredientList` y su objeto `Name authorName`. Si una instancia de `Recipe` es destruida de la memoria, su vector interno de ingredientes y su objeto `Name` se destruyen automáticamente con ella. |
| **Agregación** (Rombo blanco hueco ◇) | • `RecipeUI` (1) ◇— (0..*) `Recipe`<br>• `RecipeUI` (1) ◇— (0..*) `Chef` | La ventana principal `RecipeUI` administra el catálogo de recetas (mediante referencia al `std::vector<Recipe>` principal) y la colección de chefs registrados (`std::vector<Chef>`), cuyos datos persisten en archivos externos más allá del ciclo de vida de la ventana. |
| **Asociación / Colaboración** (Flecha abierta →) | • `Recipe` (0..*) —→ (1) `Chef`<br>• `RecipeUI` —→ `IngredientUI` | El método `Recipe::asignarChefAutor(const Chef&)` permite que una receta colabore con un objeto `Chef` existente para tomar su identidad como autor sin ser propietaria del ciclo de vida del chef. Asimismo, `RecipeUI` instancia y delega en `IngredientUI` la edición modal de ingredientes. |

### 3.4 Diagrama de Casos de Uso UML

En la **Figura 2** se muestra el Diagrama de Casos de Uso UML que modela las interacciones del actor (*Usuario* / *Chef*) con los 7 casos de uso principales del sistema gráfico:

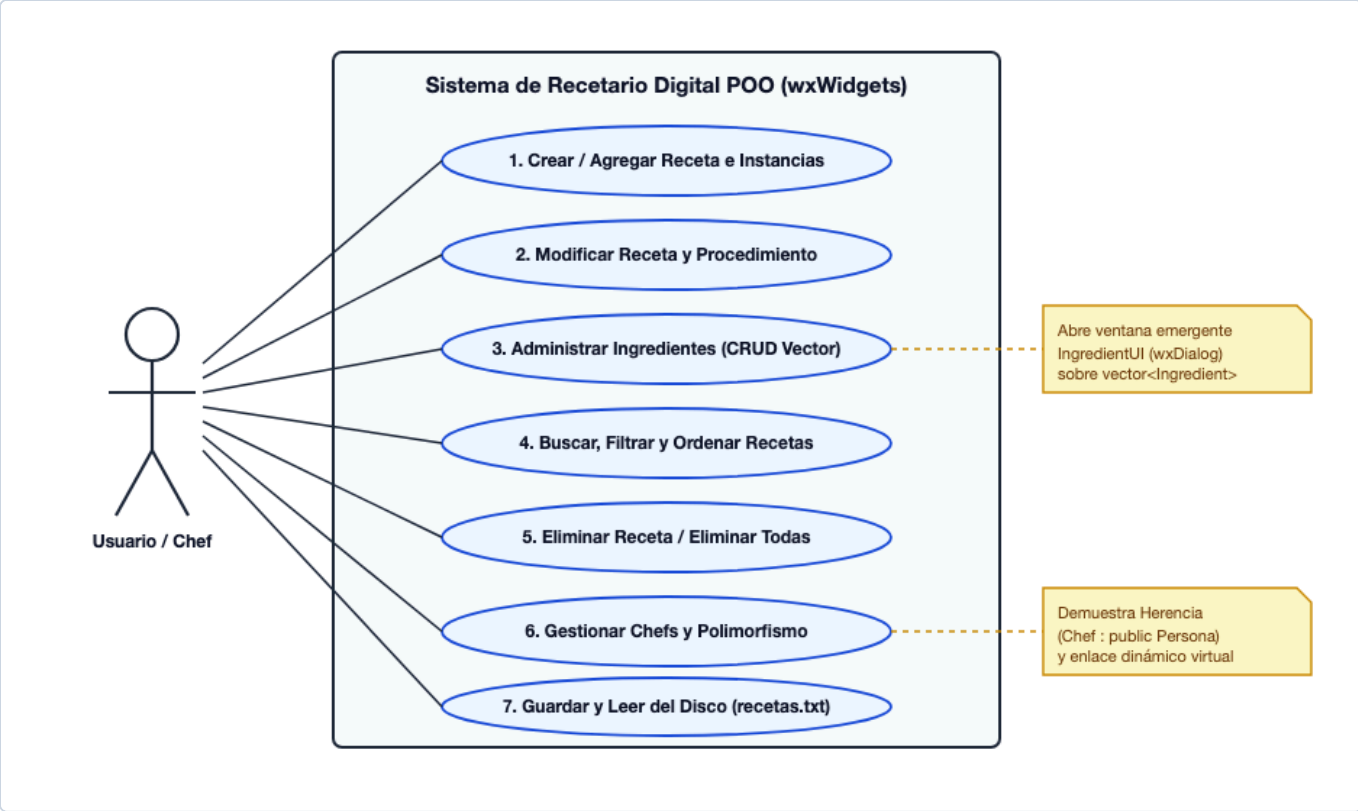



*Figura 2. Diagrama de Casos de Uso UML del Sistema de Recetario Digital.*

## 4. Implementación del Sistema (Código Fuente C++17 y wxWidgets)

El proyecto conserva la estructura modular de carpetas del proyecto preliminar de Estructuras de Datos (`include/` para cabeceras y `src/` para implementaciones), adaptando las colecciones a `std::vector` e integrando las clases de POO y la interfaz gráfica con **wxWidgets**. A continuación, se presenta el código fuente íntegro y documentado de todos los módulos que conforman la aplicación:

**Archivo: `include/name.hpp` — Cabecera de la clase Name (Abstracción de Nombre y Apellido)**

```
#ifndef __NAME_H__
#define __NAME_H__

#include <string>
#include <istream>
#include <ostream>
#include <sstream>

class Name {
private:
    std::string first;
    std::string last;

public:
    // Constructores y destructor
    Name();
    Name(const std::string& f, const std::string& l);
    Name(const Name&);
    ~Name();

    // Getters
    std::string getFirst() const;
    std::string getLast() const;

    // Setters
    void setFirst(const std::string&);
    void setLast(const std::string&);

    std::string toString() const;

    Name& operator=(const Name&);

    // Sobrecarga de operadores
    bool operator==(const Name&) const;
    bool operator!=(const Name&) const;
    bool operator<(const Name&) const;
    bool operator<=(const Name&) const;
    bool operator>(const Name&) const;
    bool operator>=(const Name&) const;

    // Comparadores
    int compare(const Name&) const;
    static int compare(const Name&, const Name&);
    static int compareByFirst(const Name&, const Name&);
    static int compareByLast(const Name&, const Name&);

    // Operadores de flujo
    friend std::ostream& operator<<(std::ostream&, const Name&);
```

```
    friend std::istream& operator>>(std::istream&, Name&);

    // Serializacion y deserializacion
    std::string serialize(const char& delim = '*') const;
    void deserialize(const std::string&, const char& delim = '*');
};

#endif // __NAME_H__
```

**Archivo: `src/name.cpp` — Implementación de la clase Name**

```
#include "name.hpp"
using namespace std;

Name::Name() : first(""), last("") { }

Name::Name(const std::string& f, const std::string& l) : first(f), last(l) { }

Name::Name(const Name& other) : first(other.first), last(other.last) { }

Name::~Name() { }

std::string Name::getFirst() const {
    return this->first;
}

std::string Name::getLast() const {
    return this->last;
}

void Name::setFirst(const std::string& f) {
    this->first = f;
}

void Name::setLast(const std::string& l) {
    this->last = l;
}

std::string Name::toString() const {
    if (this->first.empty() && this->last.empty()) {
        return "";
    }
    return this->first + " " + this->last;
}

Name& Name::operator=(const Name& other) {
    this->first = other.first;
    this->last = other.last;
    return *this;
}

bool Name::operator==(const Name& other) const {
    return (this->first == other.first) and (this->last == other.last);
}

bool Name::operator!=(const Name& other) const {
    return !(*this == other);
}

bool Name::operator<(const Name& other) const {
```

```
    if (this->first == other.first) {
        return this->last < other.last;
    }
    return this->first < other.first;
}

bool Name::operator<=(const Name& other) const {
    return (*this < other) or (*this == other);
}

bool Name::operator>(const Name& other) const {
    return !(*this <= other);
}

bool Name::operator>=(const Name& other) const {
    return !(*this < other);
}

int Name::compare(const Name& other) const {
    int res = this->first.compare(other.first);
    if (res == 0) {
        return this->last.compare(other.last);
    }
    return res;
}

int Name::compare(const Name& a, const Name& b) {
    return a.compare(b);
}

int Name::compareByFirst(const Name& a, const Name& b) {
    return a.first.compare(b.first);
}

int Name::compareByLast(const Name& a, const Name& b) {
    return a.last.compare(b.last);
}

string Name::serialize(const char& delim) const {
    string result;
    result += this->last + delim;
    result += this->first;
    return result;
}

void Name::deserialize(const string& str, const char& delim) {
    stringstream mySStream(str);
    getline(mySStream, this->last, delim);
    getline(mySStream, this->first);
}

istream& operator>>(istream& is, Name& i) {
    if (!getline(is, i.first)) return is;
    getline(is, i.last);
    return is;
}

ostream& operator<<(ostream& os, const Name& i) {
    os << i.first << endl;
    os << i.last << endl;
```

```
    return os;
}
```

**Archivo: `include/ingredient.hpp` — Cabecera de la clase Ingredient (Abstracción de Ingrediente)**

```
#ifndef __INGREDIENT_H__
#define __INGREDIENT_H__

#include <string>
#include <iostream>
#include <fstream>
#include <sstream>
#include <iomanip>
#include <cstdio>

inline const int wIngredientName = 23;
inline const int wQuantity = 16;
inline const int wUnit = 10;

class Ingredient {
private:
    std::string name;
    float quantity;
    std::string unit;

public:
    // Constructores
    Ingredient();
    Ingredient(const std::string& n, const float& q, const std::string& u);
    Ingredient(const Ingredient& other);

    // Getters
    std::string getName() const;
    float getQuantity() const;
    std::string getUnit() const;

    // Setters
    void setName(const std::string&);
    void setQuantity(const float&);
    void setUnit(const std::string&);

    // To string
    std::string toString() const;

    // Sobrecarga de operadores
    Ingredient& operator=(const Ingredient&);
    bool operator==(const Ingredient&) const;
    bool operator!=(const Ingredient&) const;
    bool operator<=(const Ingredient&) const;
    bool operator<(const Ingredient&) const;
    bool operator>(const Ingredient&) const;
    bool operator>=(const Ingredient&) const;

    int compare(const Ingredient&) const;
    static int compare(const Ingredient&, const Ingredient&);
    static int compareByName(const Ingredient&, const Ingredient&);
    static int compareByQuantity(const Ingredient&, const Ingredient&);
    static int compareByUnit(const Ingredient&, const Ingredient&);

    // Sobrecarga de operadores de flujo
```

```
    friend std::ostream& operator<<(std::ostream&, const Ingredient&);
    friend std::istream& operator>>(std::istream&, Ingredient&);

    std::string serialize(const char& delim = '*') const;
    void deserialize(const std::string&, const char& delim = '*');
};

#endif // __INGREDIENT_H__
```

**Archivo: `src/ingredient.cpp` — Implementación de la clase Ingredient**

```
#include "ingredient.hpp"
using namespace std;

// Constructores
Ingredient::Ingredient() : name(""), quantity(0.0f), unit("") { }

Ingredient::Ingredient(const std::string& n, const float& q, const std::string& u)
    : name(n), quantity(q), unit(u) { }

Ingredient::Ingredient(const Ingredient& other)
    : name(other.name), quantity(other.quantity), unit(other.unit) { }

// Getters
string Ingredient::getName() const {
    return this->name;
}

float Ingredient::getQuantity() const {
    return this->quantity;
}

string Ingredient::getUnit() const {
    return this->unit;
}

// Setters
void Ingredient::setName(const string& n) {
    this->name = n;
}

void Ingredient::setQuantity(const float& q) {
    this->quantity = q;
}

void Ingredient::setUnit(const string& u) {
    this->unit = u;
}

string Ingredient::toString() const {
    stringstream ss;

    ss << " | " << left << setw(wIngredientName) << this->name << " | "
       << left << setw(wQuantity) << to_string(this->quantity) << " | "
       << left << setw(wUnit) << this->unit << " | "
       << endl;

    return ss.str();
}
```

```
// Sobrecarga de operador de asignacion
Ingredient& Ingredient::operator=(const Ingredient& other) {
    this->name = other.name;
    this->quantity = other.quantity;
    this->unit = other.unit;

    return *this;
}

// Operadores
bool Ingredient::operator==(const Ingredient& other) const {
    return this->name == other.name;
}

bool Ingredient::operator!=(const Ingredient& other) const {
    return this->name != other.name;
}

bool Ingredient::operator<=(const Ingredient& other) const {
    return this->name <= other.name;
}

bool Ingredient::operator<(const Ingredient& other) const {
    return this->name < other.name;
}

bool Ingredient::operator>(const Ingredient& other) const {
    return this->name > other.name;
}

bool Ingredient::operator>=(const Ingredient& other) const {
    return this->name >= other.name;
}

// Comparaciones
int Ingredient::compare(const Ingredient& other) const {
    return this->name.compare(other.name);
}

int Ingredient::compare(const Ingredient& a, const Ingredient& b) {
    return a.compare(b);
}

int Ingredient::compareByName(const Ingredient& a, const Ingredient& b) {
    return a.compare(b);
}

int Ingredient::compareByQuantity(const Ingredient& a, const Ingredient& b) {
    if (a.quantity < b.quantity) {
        return -1;
    } else if (a.quantity > b.quantity) {
        return 1;
    } else {
        return 0;
    }
}

int Ingredient::compareByUnit(const Ingredient& a, const Ingredient& b) {
    return a.unit.compare(b.unit);
}

std::string Ingredient::serialize(const char& delim) const {
```

```
    string result;
    char myQuantity[20];
    snprintf(myQuantity, sizeof(myQuantity), "%.3f", this->quantity);

    result += this->name + delim;
    result += myQuantity;
    result += delim;
    result += this->unit;

    return result;
}

void Ingredient::deserialize(const std::string& str, const char& delim) {
    stringstream mySStream(str);
    string myStr;

    getline(mySStream, this->name, delim);
    getline(mySStream, myStr, delim);
    try {
        this->quantity = myStr.empty() ? 0.0f : stof(myStr);
    } catch (...) {
        this->quantity = 0.0f;
    }
    getline(mySStream, this->unit, delim);
}

// Sobrecarga de operadores de flujo
istream& operator>>(istream& is, Ingredient& i) {
    string myStr;

    if (!getline(is, i.name)) return is;

    getline(is, myStr);
    try {
        i.quantity = stof(myStr);
    } catch (...) {
        i.quantity = 0.0f;
    }

    getline(is, i.unit);

    return is;
}

ostream& operator<<(ostream& os, const Ingredient& i) {
    os << i.name << endl;
    os << i.quantity << endl;
    os << i.unit << endl;
    return os;
}
```

**Archivo: `include/chef.hpp` — Cabecera de la interfaz IMostrable, clase base Persona y clase derivada Chef**

```
#ifndef __CHEF_H__
#define __CHEF_H__

#include <string>
#include <sstream>
#include "name.hpp"
```

```
// Interfaz <<interface>> para demostrar Realizacion y Abstraccion pura
class IMostrable {
public:
    virtual std::string obtenerFicha() const = 0;
    virtual ~IMostrable() = default;
};

// Clase Base Persona (Demuestra Composicion con Name, Encapsulamiento y Herencia)
class Persona : public IMostrable {
protected:
    Name nombre;
    int edad;

public:
    Persona();
    Persona(const Name& n, int e);
    Persona(const Persona& other);
    virtual ~Persona();

    // Getters y Setters
    Name getNombre() const;
    int getEdad() const;

    void setNombre(const Name& n);
    void setEdad(int e);

    // Metodos virtuales para Polimorfismo
    virtual std::string obtenerRol() const;
    virtual std::string obtenerFicha() const override;
};

// Clase Derivada Chef : "Un Chef ES UNA Persona" (Herencia y Polimorfismo)
class Chef : public Persona {
private:
    std::string especialidad;
    int aniosExperiencia;

public:
    Chef();
    Chef(const Name& n, int e, const std::string& esp, int exp);
    Chef(const Chef& other);
    virtual ~Chef();

    // Getters
    std::string getEspecialidad() const;
    int getAniosExperiencia() const;

    // Setters
    void setEspecialidad(const std::string& esp);
    void setAniosExperiencia(int exp);

    // Sobrescritura polimorfica (override)
    std::string obtenerRol() const override;
    std::string obtenerFicha() const override;

    std::string toString() const;

    Chef& operator=(const Chef& other);
    bool operator==(const Chef& other) const;

    // Serializacion para persistencia en disco
```

```
    std::string serialize(const char& delim = '*') const;
    void deserialize(const std::string& str, const char& delim = '*');
};

// Funcion que demuestra polimorfismo por sustitucion (enlace dinamico)
std::string consultarAutorPolimorfico(const Persona& persona);

#endif // __CHEF_H__
```

**Archivo: `src/chef.cpp` — Implementación de Persona, Chef y función polimórfica consultarAutorPolimorfico()**

```
#include "chef.hpp"
using namespace std;

// ==================== Implementacion de Persona ====================

Persona::Persona() : nombre(), edad(18) { }

Persona::Persona(const Name& n, int e) : nombre(n), edad(e) { }

Persona::Persona(const Persona& other) : nombre(other.nombre), edad(other.edad) { }

Persona::~Persona() { }

Name Persona::getNombre() const {
    return this->nombre;
}

int Persona::getEdad() const {
    return this->edad;
}

void Persona::setNombre(const Name& n) {
    this->nombre = n;
}

void Persona::setEdad(int e) {
    this->edad = (e > 0) ? e : 18;
}

std::string Persona::obtenerRol() const {
    return "Persona General";
}

std::string Persona::obtenerFicha() const {
    stringstream ss;
    ss << "[" << obtenerRol() << "] Nombre: " << this->nombre.toString()
       << " | Edad: " << this->edad << " anios";
    return ss.str();
}

// ==================== Implementacion de Chef ====================

Chef::Chef() : Persona(), especialidad("Cocina General"), aniosExperiencia(1) { }

Chef::Chef(const Name& n, int e, const std::string& esp, int exp)
    : Persona(n, e), especialidad(esp), aniosExperiencia(exp) { }

Chef::Chef(const Chef& other)
```

```
    : Persona(other.nombre, other.edad),
      especialidad(other.especialidad),
      aniosExperiencia(other.aniosExperiencia) { }

Chef::~Chef() { }

std::string Chef::getEspecialidad() const {
    return this->especialidad;
}

int Chef::getAniosExperiencia() const {
    return this->aniosExperiencia;
}

void Chef::setEspecialidad(const std::string& esp) {
    this->especialidad = esp;
}

void Chef::setAniosExperiencia(int exp) {
    this->aniosExperiencia = (exp >= 0) ? exp : 0;
}

std::string Chef::obtenerRol() const {
    return "Chef Autor";
}

std::string Chef::obtenerFicha() const {
    stringstream ss;
    ss << "[" << obtenerRol() << "] " << this->nombre.toString()
       << " | Edad: " << this->edad
       << " | Especialidad: " << this->especialidad
       << " | Experiencia: " << this->aniosExperiencia << " anios";
    return ss.str();
}

std::string Chef::toString() const {
    return this->obtenerFicha();
}

Chef& Chef::operator=(const Chef& other) {
    this->nombre = other.nombre;
    this->edad = other.edad;
    this->especialidad = other.especialidad;
    this->aniosExperiencia = other.aniosExperiencia;
    return *this;
}

bool Chef::operator==(const Chef& other) const {
    return this->nombre == other.nombre;
}

std::string Chef::serialize(const char& delim) const {
    string result;
    result += this->nombre.serialize(',') + delim;
    result += to_string(this->edad) + delim;
    result += this->especialidad + delim;
    result += to_string(this->aniosExperiencia);
    return result;
}

void Chef::deserialize(const std::string& str, const char& delim) {
    stringstream mySStream(str);
```

```
    string myStr;

    getline(mySStream, myStr, delim);
    this->nombre.deserialize(myStr, ',');

    getline(mySStream, myStr, delim);
    try {
        this->edad = myStr.empty() ? 18 : stoi(myStr);
    } catch (...) {
        this->edad = 18;
    }

    getline(mySStream, this->especialidad, delim);

    getline(mySStream, myStr, delim);
    try {
        this->aniosExperiencia = myStr.empty() ? 1 : stoi(myStr);
    } catch (...) {
        this->aniosExperiencia = 1;
    }
}

std::string consultarAutorPolimorfico(const Persona& persona) {
    return persona.obtenerFicha();
}
```

**Archivo: `include/recipe.hpp` — Cabecera de la clase Recipe (Composición con Name y std::vector<Ingredient>)**

```
#ifndef __RECIPE_H__
#define __RECIPE_H__

#include <string>
#include <vector>
#include <sstream>
#include <iomanip>
#include "ingredient.hpp"
#include "name.hpp"
#include "chef.hpp"

// Anchos para formateo del texto
inline const int wRecipeName = 30;
inline const int wAuthorName = 20;
inline const int wPrepTime = 25;
inline const int wCategory = 10;

class Recipe {
private:
    std::string name;
    Name authorName;
    int prepTime;
    std::string procedure;
    std::string category;
    std::vector<Ingredient> ingredientList;

public:
    // Constructores y destructor
    Recipe();
    Recipe(const Recipe&);
    ~Recipe();
```

```
    // Getters
    std::string getName() const;
    Name getAuthorName() const;
    int getPrepTime() const;
    std::string getProcedure() const;
    std::string getCategory() const;
    std::vector<Ingredient>& getIngredientList();
    const std::vector<Ingredient>& getIngredientList() const;

    // Setters
    void setName(const std::string&);
    void setAuthor(const Name&);
    void setPrepTime(const int&);
    void setProcedure(const std::string&);
    void setCategory(const std::string&);

    // Relacion de Asociacion / Colaboracion con la clase Chef
    void asignarChefAutor(const Chef& chef);

    // Metodos auxiliares para gestionar el vector de ingredientes
    void addIngredient(const Ingredient& ing);
    bool removeIngredientByIndex(int index);

    std::string toString(const bool s = false) const;

    Recipe& operator=(const Recipe&);

    // Sobrecarga de operadores
    bool operator==(const Recipe&) const;
    bool operator!=(const Recipe&) const;
    bool operator<(const Recipe&) const;
    bool operator<=(const Recipe&) const;
    bool operator>(const Recipe&) const;
    bool operator>=(const Recipe&) const;

    // Compare
    int compare(const Recipe&) const;
    static int compare(const Recipe&, const Recipe&);
    static int compareByName(const Recipe&, const Recipe&);
    static int compareByPrepTime(const Recipe&, const Recipe&);
    static int compareByCategory(const Recipe&, const Recipe&);
    static int compareByAuthor(const Recipe&, const Recipe&);

    // Serializacion
    std::string serialize(const char& delim = '*') const;
    void deserialize(const std::string&, const char& delim = '*');
};

#endif // __RECIPE_H__
```

**Archivo: `src/recipe.cpp` — Implementación de la clase Recipe y colaboración con Chef**

```
#include "recipe.hpp"
#include <algorithm>
using namespace std;

Recipe::Recipe() : name(""), authorName(), prepTime(0), procedure(""), category("Comida") { }

Recipe::Recipe(const Recipe& other)
```

```
    : name(other.name),
      authorName(other.authorName),
      prepTime(other.prepTime),
      procedure(other.procedure),
      category(other.category),
      ingredientList(other.ingredientList) { }

Recipe::~Recipe() { }

std::string Recipe::getName() const {
    return this->name;
}

Name Recipe::getAuthorName() const {
    return this->authorName;
}

int Recipe::getPrepTime() const {
    return this->prepTime;
}

std::string Recipe::getProcedure() const {
    return this->procedure;
}

std::string Recipe::getCategory() const {
    return this->category;
}

std::vector<Ingredient>& Recipe::getIngredientList() {
    return this->ingredientList;
}

const std::vector<Ingredient>& Recipe::getIngredientList() const {
    return this->ingredientList;
}

void Recipe::setName(const std::string& n) {
    this->name = n;
}

void Recipe::setAuthor(const Name& a) {
    this->authorName = a;
}

void Recipe::setPrepTime(const int& p) {
    this->prepTime = p;
}

void Recipe::setProcedure(const std::string& pro) {
    this->procedure = pro;
}

void Recipe::setCategory(const std::string& c) {
    this->category = c;
}

void Recipe::asignarChefAutor(const Chef& chef) {
    this->authorName = chef.getNombre();
}

void Recipe::addIngredient(const Ingredient& ing) {
```

```
    this->ingredientList.push_back(ing);
    // Mantiene los ingredientes ordenados por nombre como en el proyecto original
    std::sort(this->ingredientList.begin(), this->ingredientList.end());
}

bool Recipe::removeIngredientByIndex(int index) {
    if (index < 0 || index >= static_cast<int>(this->ingredientList.size())) {
        return false;
    }
    this->ingredientList.erase(this->ingredientList.begin() + index);
    return true;
}

std::string Recipe::toString(const bool s) const {
    stringstream ss;

    ss << " | " << left << setw(wRecipeName) << this->name << " | "
       << left << setw(wAuthorName) << this->authorName.toString() << " | "
       << left << setw(wPrepTime) << to_string(this->prepTime) << " | "
       << left << setw(wCategory) << this->category << " | "
       << endl;

    if (s) {
        ss << endl;
        ss << " | " << left << setw(wIngredientName) << "INGREDIENTE";
        ss << " | " << left << setw(wQuantity) << "CANTIDAD";
        ss << " | " << left << setw(wUnit) << "UNIDAD";
        ss << " | " << endl << "\n";

        for (size_t i = 0; i < this->ingredientList.size(); i++) {
            ss << this->ingredientList[i].toString();
        }
        ss << "\n";
    }
    ss << "Procedimiento: " << this->procedure << endl;
    return ss.str();
}

Recipe& Recipe::operator=(const Recipe& other) {
    this->name = other.name;
    this->authorName = other.authorName;
    this->prepTime = other.prepTime;
    this->procedure = other.procedure;
    this->category = other.category;
    this->ingredientList = other.ingredientList;

    return *this;
}

bool Recipe::operator==(const Recipe& other) const {
    return this->name == other.name;
}

bool Recipe::operator!=(const Recipe& other) const {
    return this->name != other.name;
}

bool Recipe::operator<(const Recipe& other) const {
    return this->name < other.name;
}

bool Recipe::operator<=(const Recipe& other) const {
```

```
    return this->name <= other.name;
}

bool Recipe::operator>(const Recipe& other) const {
    return this->name > other.name;
}

bool Recipe::operator>=(const Recipe& other) const {
    return this->name >= other.name;
}

int Recipe::compare(const Recipe& other) const {
    return this->name.compare(other.name);
}

int Recipe::compare(const Recipe& a, const Recipe& b) {
    return a.compare(b);
}

int Recipe::compareByName(const Recipe& a, const Recipe& b) {
    return a.compare(b);
}

int Recipe::compareByPrepTime(const Recipe& a, const Recipe& b) {
    if (a.prepTime > b.prepTime) return 1;
    else if (a.prepTime == b.prepTime) return 0;
    else return -1;
}

int Recipe::compareByCategory(const Recipe& a, const Recipe& b) {
    return a.category.compare(b.category);
}

int Recipe::compareByAuthor(const Recipe& a, const Recipe& b) {
    return a.authorName.compare(b.authorName);
}

std::string Recipe::serialize(const char& delim) const {
    string result;

    result += this->name + delim;
    result += this->authorName.serialize(',') + delim;
    result += to_string(this->prepTime) + delim;
    result += this->procedure + delim;
    result += this->category + delim;

    for (size_t i = 0; i < this->ingredientList.size(); i++) {
        result += this->ingredientList[i].serialize('*') + '#';
    }

    return result;
}

void Recipe::deserialize(const string& str, const char& delim) {
    stringstream mySStream(str);
    string myStr;

    getline(mySStream, this->name, delim);
    getline(mySStream, myStr, delim);
    this->authorName.deserialize(myStr, ',');
    getline(mySStream, myStr, delim);
    try {
```

```
        this->prepTime = myStr.empty() ? 0 : static_cast<int>(stof(myStr));
    } catch (...) {
        this->prepTime = 0;
    }
    getline(mySStream, this->procedure, delim);
    getline(mySStream, this->category, delim);
    getline(mySStream, myStr);

    this->ingredientList.clear();
    stringstream ingStream(myStr);
    string ingToken;
    while (getline(ingStream, ingToken, '#')) {
        if (ingToken.empty()) {
            continue;
        }
        Ingredient myIng;
        myIng.deserialize(ingToken, '*');
        this->ingredientList.push_back(myIng);
    }
}
```

**Archivo: `include/ingredientui.hpp` — Cabecera del diálogo gráfico IngredientUI (hereda de wxDialog)**

```
#ifndef __INGREDIENTUI_H__
#define __INGREDIENTUI_H__

#include <wx/wx.h>
#include <wx/listctrl.h>
#include <vector>
#include <string>
#include <algorithm>
#include "ingredient.hpp"

class IngredientUI : public wxDialog {
private:
    std::vector<Ingredient>& ingredientListRef;
    int selectedIndex;

    wxListCtrl* listCtrl;
    wxTextCtrl* txtName;
    wxTextCtrl* txtQuantity;
    wxTextCtrl* txtUnit;

    void refreshTable();
    void onSelectIngredient(wxListEvent& event);
    void onAddIngredient(wxCommandEvent& event);
    void onModifyQuantity(wxCommandEvent& event);
    void onDeleteIngredient(wxCommandEvent& event);
    void onDeleteAllIngredients(wxCommandEvent& event);
    void onCloseDialog(wxCommandEvent& event);

public:
    IngredientUI(wxWindow* parent, std::vector<Ingredient>& list, const std::string& recipeTitle =
"");
    void manageIngredient();
};

#endif // __INGREDIENTUI_H__
```

**Archivo: `src/ingredientui.cpp` — Implementación del diálogo gráfico IngredientUI para gestionar el vector de ingredientes**

```
#include "ingredientui.hpp"
#include <sstream>
#include <iomanip>

using namespace std;

IngredientUI::IngredientUI(wxWindow* parent, std::vector<Ingredient>& list, const std::string&
recipeTitle)
    : wxDialog(parent, wxID_ANY,
               wxString::FromUTF8(recipeTitle.empty()
                   ? "Gestion de Ingredientes"
                   : "Gestion de Ingredientes - " + recipeTitle),
               wxDefaultPosition, wxSize(640, 480),
               wxDEFAULT_DIALOG_STYLE | wxRESIZE_BORDER),
      ingredientListRef(list),
      selectedIndex(-1) {

    wxBoxSizer* mainSizer = new wxBoxSizer(wxVERTICAL);

    // Encabezado
    wxStaticText* lblHeader = new wxStaticText(
        this, wxID_ANY,
        wxString::FromUTF8("Administracion de Ingredientes de la Receta
(std::vector<Ingredient>)"));
    wxFont headerFont = lblHeader->GetFont();
    headerFont.SetPointSize(12);
    headerFont.SetWeight(wxFONTWEIGHT_BOLD);
    lblHeader->SetFont(headerFont);
    mainSizer->Add(lblHeader, 0, wxALL, 10);

    // Tabla de ingredientes
    listCtrl = new wxListCtrl(this, wxID_ANY, wxDefaultPosition, wxSize(-1, 200),
                              wxLC_REPORT | wxLC_SINGLE_SEL | wxBORDER_SUNKEN);
    listCtrl->InsertColumn(0, "Pos", wxLIST_FORMAT_LEFT, 55);
    listCtrl->InsertColumn(1, "Ingrediente", wxLIST_FORMAT_LEFT, 240);
    listCtrl->InsertColumn(2, "Cantidad", wxLIST_FORMAT_LEFT, 130);
    listCtrl->InsertColumn(3, "Unidad", wxLIST_FORMAT_LEFT, 160);
    mainSizer->Add(listCtrl, 1, wxEXPAND | wxLEFT | wxRIGHT | wxBOTTOM, 10);

    // Formulario de captura / edicion
    wxStaticBoxSizer* formBox = new wxStaticBoxSizer(wxVERTICAL, this, "Datos del Ingrediente");
    wxFlexGridSizer* grid = new wxFlexGridSizer(3, 2, 8, 12);
    grid->AddGrowableCol(1, 1);

    grid->Add(new wxStaticText(this, wxID_ANY, "Nombre del ingrediente:"), 0,
wxALIGN_CENTER_VERTICAL);
    txtName = new wxTextCtrl(this, wxID_ANY);
    grid->Add(txtName, 1, wxEXPAND);

    grid->Add(new wxStaticText(this, wxID_ANY, "Cantidad:"), 0, wxALIGN_CENTER_VERTICAL);
    txtQuantity = new wxTextCtrl(this, wxID_ANY);
    grid->Add(txtQuantity, 1, wxEXPAND);

    grid->Add(new wxStaticText(this, wxID_ANY, "Unidad (Gramos, Kg, Lt, Piezas...):"), 0,
wxALIGN_CENTER_VERTICAL);
    txtUnit = new wxTextCtrl(this, wxID_ANY);
    grid->Add(txtUnit, 1, wxEXPAND);
```

```
    formBox->Add(grid, 1, wxEXPAND | wxALL, 8);
    mainSizer->Add(formBox, 0, wxEXPAND | wxLEFT | wxRIGHT | wxBOTTOM, 10);

    // Botones de accion (mismas opciones que el menu original de IngredientUI)
    wxBoxSizer* btnSizer = new wxBoxSizer(wxHORIZONTAL);
    wxButton* btnAdd = new wxButton(this, wxID_ANY, "1. Agregar");
    wxButton* btnModify = new wxButton(this, wxID_ANY, "2. Modificar Cantidad");
    wxButton* btnDelete = new wxButton(this, wxID_ANY, "3. Eliminar");
    wxButton* btnDeleteAll = new wxButton(this, wxID_ANY, "4. Borrar Todos");
    wxButton* btnClose = new wxButton(this, wxID_OK, "5. Guardar y Salir");

    btnSizer->Add(btnAdd, 0, wxRIGHT, 6);
    btnSizer->Add(btnModify, 0, wxRIGHT, 6);
    btnSizer->Add(btnDelete, 0, wxRIGHT, 6);
    btnSizer->Add(btnDeleteAll, 0, wxRIGHT, 6);
    btnSizer->AddStretchSpacer(1);
    btnSizer->Add(btnClose, 0);

    mainSizer->Add(btnSizer, 0, wxEXPAND | wxLEFT | wxRIGHT | wxBOTTOM, 10);

    SetSizer(mainSizer);
    CentreOnParent();

    // Eventos
    listCtrl->Bind(wxEVT_LIST_ITEM_SELECTED, &IngredientUI::onSelectIngredient, this);
    btnAdd->Bind(wxEVT_BUTTON, &IngredientUI::onAddIngredient, this);
    btnModify->Bind(wxEVT_BUTTON, &IngredientUI::onModifyQuantity, this);
    btnDelete->Bind(wxEVT_BUTTON, &IngredientUI::onDeleteIngredient, this);
    btnDeleteAll->Bind(wxEVT_BUTTON, &IngredientUI::onDeleteAllIngredients, this);
    btnClose->Bind(wxEVT_BUTTON, &IngredientUI::onCloseDialog, this);

    refreshTable();
    if (!this->ingredientListRef.empty()) {
        selectedIndex = 0;
        const Ingredient& ing = this->ingredientListRef[0];
        txtName->SetValue(wxString::FromUTF8(ing.getName()));
        stringstream ss;
        ss << fixed << setprecision(2) << ing.getQuantity();
        txtQuantity->SetValue(ss.str());
        txtUnit->SetValue(wxString::FromUTF8(ing.getUnit()));
    }
}

void IngredientUI::manageIngredient() {
    ShowModal();
}

void IngredientUI::refreshTable() {
    listCtrl->DeleteAllItems();
    for (size_t i = 0; i < this->ingredientListRef.size(); i++) {
        const Ingredient& ing = this->ingredientListRef[i];
        long idx = listCtrl->InsertItem(i, to_string(i));
        listCtrl->SetItem(idx, 1, wxString::FromUTF8(ing.getName()));

        stringstream ss;
        ss << fixed << setprecision(2) << ing.getQuantity();
        listCtrl->SetItem(idx, 2, ss.str());
        listCtrl->SetItem(idx, 3, wxString::FromUTF8(ing.getUnit()));
    }
    listCtrl->Refresh();
    listCtrl->Update();
}
```

```
void IngredientUI::onSelectIngredient(wxListEvent& event) {
    selectedIndex = event.GetIndex();
    if (selectedIndex >= 0 && selectedIndex < static_cast<int>(this->ingredientListRef.size())) {
        const Ingredient& ing = this->ingredientListRef[selectedIndex];
        txtName->SetValue(wxString::FromUTF8(ing.getName()));
        stringstream ss;
        ss << fixed << setprecision(2) << ing.getQuantity();
        txtQuantity->SetValue(ss.str());
        txtUnit->SetValue(wxString::FromUTF8(ing.getUnit()));
    }
}

void IngredientUI::onAddIngredient(wxCommandEvent&) {
    string nameStr = txtName->GetValue().ToUTF8().data();
    string qtyStr = txtQuantity->GetValue().ToUTF8().data();
    string unitStr = txtUnit->GetValue().ToUTF8().data();

    if (nameStr.empty() || qtyStr.empty() || unitStr.empty()) {
        wxMessageBox("Por favor llena el nombre, la cantidad y la unidad del ingrediente.",
                     "Campos incompletos", wxOK | wxICON_WARNING, this);
        return;
    }

    float qty = 0.0f;
    try {
        qty = stof(qtyStr);
    } catch (...) {
        wxMessageBox("La cantidad debe ser un numero valido.", "Error", wxOK | wxICON_ERROR, this);
        return;
    }

    Ingredient tempIng;
    tempIng.setName(nameStr);
    tempIng.setQuantity(qty);
    tempIng.setUnit(unitStr);

    this->ingredientListRef.push_back(tempIng);
    std::sort(this->ingredientListRef.begin(), this->ingredientListRef.end());

    txtName->Clear();
    txtQuantity->Clear();
    txtUnit->Clear();
    selectedIndex = -1;
    refreshTable();
    if (!this->ingredientListRef.empty()) {
        selectedIndex = 0;
        const Ingredient& ing = this->ingredientListRef[0];
        txtName->SetValue(wxString::FromUTF8(ing.getName()));
        stringstream ss;
        ss << fixed << setprecision(2) << ing.getQuantity();
        txtQuantity->SetValue(ss.str());
        txtUnit->SetValue(wxString::FromUTF8(ing.getUnit()));
    }
}

void IngredientUI::onModifyQuantity(wxCommandEvent&) {
    string nameStr = txtName->GetValue().ToUTF8().data();
    string qtyStr = txtQuantity->GetValue().ToUTF8().data();
    string unitStr = txtUnit->GetValue().ToUTF8().data();

    int pos = selectedIndex;
```

```
    if (pos < 0 || pos >= static_cast<int>(this->ingredientListRef.size())) {
        // Buscar por nombre si no selecciono con clic
        for (size_t i = 0; i < this->ingredientListRef.size(); i++) {
            if (this->ingredientListRef[i].getName() == nameStr) {
                pos = static_cast<int>(i);
                break;
            }
        }
    }

    if (pos < 0 || pos >= static_cast<int>(this->ingredientListRef.size())) {
        wxMessageBox("Selecciona un ingrediente de la tabla o escribe su nombre exacto para
modificarlo.",
                     "Ingrediente no encontrado", wxOK | wxICON_WARNING, this);
        return;
    }

    float qty = 0.0f;
    try {
        qty = stof(qtyStr);
    } catch (...) {
        wxMessageBox("Escribe una cantidad numerica valida.", "Error", wxOK | wxICON_ERROR, this);
        return;
    }

    if (!nameStr.empty()) {
        this->ingredientListRef[pos].setName(nameStr);
    }
    this->ingredientListRef[pos].setQuantity(qty);
    if (!unitStr.empty()) {
        this->ingredientListRef[pos].setUnit(unitStr);
    }

    refreshTable();
    wxMessageBox("Ingrediente modificado con exito!", "Actualizado", wxOK | wxICON_INFORMATION,
this);
}

void IngredientUI::onDeleteIngredient(wxCommandEvent&) {
    string nameStr = txtName->GetValue().ToUTF8().data();
    int pos = selectedIndex;

    if (pos < 0 || pos >= static_cast<int>(this->ingredientListRef.size())) {
        for (size_t i = 0; i < this->ingredientListRef.size(); i++) {
            if (this->ingredientListRef[i].getName() == nameStr) {
                pos = static_cast<int>(i);
                break;
            }
        }
    }

    if (pos < 0 || pos >= static_cast<int>(this->ingredientListRef.size())) {
        wxMessageBox("Selecciona un ingrediente de la tabla para eliminarlo.",
                     "Sin seleccion", wxOK | wxICON_WARNING, this);
        return;
    }

    string targetName = this->ingredientListRef[pos].getName();
    int resp = wxMessageBox(
        wxString::FromUTF8("¿Deseas eliminar el ingrediente '" + targetName + "'?"),
        "Confirmar eliminacion", wxYES_NO | wxICON_QUESTION, this);
```

```
    if (resp == wxYES) {
        this->ingredientListRef.erase(this->ingredientListRef.begin() + pos);
        selectedIndex = -1;
        txtName->Clear();
        txtQuantity->Clear();
        txtUnit->Clear();
        refreshTable();
    }
}

void IngredientUI::onDeleteAllIngredients(wxCommandEvent&) {
    if (this->ingredientListRef.empty()) {
        wxMessageBox("La lista de ingredientes ya esta vacia.", "Aviso", wxOK | wxICON_INFORMATION,
this);
        return;
    }

    int resp = wxMessageBox("¿Estas seguro de eliminar TODOS los ingredientes de esta receta?",
                            "Confirmar", wxYES_NO | wxICON_WARNING, this);
    if (resp == wxYES) {
        this->ingredientListRef.clear();
        selectedIndex = -1;
        refreshTable();
    }
}

void IngredientUI::onCloseDialog(wxCommandEvent&) {
    EndModal(wxID_OK);
}
```

**Archivo: `include/recipeui.hpp` — Cabecera de la ventana principal RecipeUI (hereda de wxFrame)**

```
#ifndef __RECIPEUI_H__
#define __RECIPEUI_H__

#include <wx/wx.h>
#include <wx/listctrl.h>
#include <wx/notebook.h>
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include <algorithm>

#include "recipe.hpp"
#include "ingredient.hpp"
#include "ingredientui.hpp"
#include "name.hpp"
#include "chef.hpp"

class RecipeUI : public wxFrame {
private:
    std::vector<Recipe>& recipeListRef;
    std::vector<Chef> chefList;
    std::vector<int> displayedIndices; // Indices reales en recipeListRef mostrados en la tabla
    int selectedRecipePos;             // Posicion real dentro de recipeListRef
    int selectedChefPos;               // Posicion real dentro de chefList

    // Controles de la pestaña de Recetas
    wxNotebook* notebook;
```

```
    wxListCtrl* recipeTable;
    wxChoice* choiceFilterCategory;
    wxChoice* choiceSearchCriteria;
    wxTextCtrl* txtSearch;

    // Formulario de creacion/edicion de Receta
    wxTextCtrl* txtRecipeName;
    wxTextCtrl* txtAuthorFirst;
    wxTextCtrl* txtAuthorLast;
    wxChoice* choiceChefSelector;
    wxTextCtrl* txtPrepTime;
    wxChoice* choiceCategory;
    wxTextCtrl* txtProcedure;
    std::vector<Ingredient> tempIngredients; // Ingredientes temporales para nueva receta

    // Visor de detalle de receta seleccionada
    wxTextCtrl* txtRecipeDetail;

    // Controles de la pestaña de Chefs (Herencia Persona -> Chef)
    wxListCtrl* chefTable;
    wxTextCtrl* txtChefFirst;
    wxTextCtrl* txtChefLast;
    wxTextCtrl* txtChefAge;
    wxTextCtrl* txtChefSpecialty;
    wxTextCtrl* txtChefExp;
    wxTextCtrl* txtPolymorphismOutput;

    // Metodos internos de inicializacion y refresco
    void initDefaultChefs();
    wxPanel* createRecipesPage(wxWindow* parent);
    wxPanel* createChefsPage(wxWindow* parent);

    void refreshRecipeTable(const std::string& filterCategory = "Todas");
    void refreshChefTable();
    void updateChefChoiceSelector();
    void showRecipeDetails(int realIndex);

    // Eventos de Recetas (equivalentes a los metodos de RecipeUI en EDD)
    void onSelectRecipe(wxListEvent& event);
    void onAddRecipe(wxCommandEvent& event);
    void onModifyRecipe(wxCommandEvent& event);
    void onDeleteRecipe(wxCommandEvent& event);
    void onDeleteAllRecipes(wxCommandEvent& event);
    void onManageIngredients(wxCommandEvent& event);
    void onFindRecipe(wxCommandEvent& event);
    void onFilterCategory(wxCommandEvent& event);
    void onSortByName(wxCommandEvent& event);
    void onSortByPrepTime(wxCommandEvent& event);
    void onWriteToDisk(wxCommandEvent& event);
    void onReadFromDisk(wxCommandEvent& event);
    void onChefSelectedForRecipe(wxCommandEvent& event);
    void onClearRecipeForm(wxCommandEvent& event);

    // Eventos de Chefs (CRUD de clase Chef + Demostracion de Polimorfismo)
    void onSelectChef(wxListEvent& event);
    void onAddChef(wxCommandEvent& event);
    void onModifyChef(wxCommandEvent& event);
    void onDeleteChef(wxCommandEvent& event);
    void onDemonstratePolymorphism(wxCommandEvent& event);

public:
    RecipeUI(std::vector<Recipe>& list);
```

```
    void loadFromFile(const std::string& filename);
    bool saveToFile(const std::string& filename);
};

#endif // __RECIPEUI_H__
```

**Archivo: `src/recipeui.cpp` — Implementación de RecipeUI (Pestañas de Recetario Digital y Autores/Chefs POO)**

```
#include "recipeui.hpp"

using namespace std;

RecipeUI::RecipeUI(std::vector<Recipe>& list)
    : wxFrame(nullptr, wxID_ANY,
              wxString::FromUTF8("Recetario Digital POO - CUCEI (wxWidgets + std::vector)"),
              wxDefaultPosition, wxSize(1120, 740)),
      recipeListRef(list),
      selectedRecipePos(-1),
      selectedChefPos(-1) {

    CreateStatusBar();
    SetStatusText(wxString::FromUTF8("Bienvenido a tu Recetario Digital - Actividad Integradora
POO"));

    initDefaultChefs();

    wxBoxSizer* rootSizer = new wxBoxSizer(wxVERTICAL);
    notebook = new wxNotebook(this, wxID_ANY);

    notebook->AddPage(createRecipesPage(notebook), wxString::FromUTF8("1. Recetario Digital
(Recetas e Ingredientes)"), true);
    notebook->AddPage(createChefsPage(notebook), wxString::FromUTF8("2. Autores / Chefs (Herencia y
Polimorfismo)"), false);

    rootSizer->Add(notebook, 1, wxEXPAND | wxALL, 6);
    SetSizer(rootSizer);
    Centre();

    // Intentar cargar recetas.txt automaticamente al iniciar
    ifstream checkFile("recetas.txt");
    if (checkFile.is_open()) {
        checkFile.close();
        loadFromFile("recetas.txt");
    } else {
        refreshRecipeTable();
    }
    refreshChefTable();
    updateChefChoiceSelector();
}

void RecipeUI::initDefaultChefs() {
    chefList.push_back(Chef(Name("Ana", "Garcia"), 34, "Cocina Navidena y Tradicional", 12));
    chefList.push_back(Chef(Name("Luis", "Martinez"), 41, "Carnes y Horno", 18));
    chefList.push_back(Chef(Name("Carlos", "Hernandez"), 29, "Desayunos Mexicanos", 7));
    chefList.push_back(Chef(Name("Juan", "Perez"), 38, "Cocina Ligera y Cenas", 14));
    chefList.push_back(Chef(Name("Maria", "Lopez"), 45, "Gastronomia Rustica", 20));
    chefList.push_back(Chef(Name("Laura", "Ramirez"), 31, "Ensaladas y Sopas", 9));
    chefList.push_back(Chef(Name("Jose", "Sanchez"), 50, "Reposteria y Platillos Tipicos", 25));
    chefList.push_back(Chef(Name("David", "Torres"), 36, "Cocina Gourmet", 11));
```

```
    chefList.push_back(Chef(Name("Elena", "Flores"), 33, "Cocina Casera", 10));
}

wxPanel* RecipeUI::createRecipesPage(wxWindow* parent) {
    wxPanel* panel = new wxPanel(parent, wxID_ANY);
    wxBoxSizer* pageSizer = new wxBoxSizer(wxVERTICAL);

    // Barra superior: Busqueda, Filtro por categoria, Ordenamiento y Disco (en 2 filas para mejor
ajuste)
    wxStaticBoxSizer* topBar = new wxStaticBoxSizer(wxVERTICAL, panel, "Busqueda, Filtrado,
Ordenamiento y Archivo");

    wxBoxSizer* row1 = new wxBoxSizer(wxHORIZONTAL);
    row1->Add(new wxStaticText(panel, wxID_ANY, "Buscar por:"), 0, wxALIGN_CENTER_VERTICAL |
wxRIGHT, 6);
    wxArrayString searchChoices;
    searchChoices.Add("1. Nombre del platillo");
    searchChoices.Add("2. Nombre del autor");
    choiceSearchCriteria = new wxChoice(panel, wxID_ANY, wxDefaultPosition, wxDefaultSize,
searchChoices);
    choiceSearchCriteria->SetSelection(0);
    row1->Add(choiceSearchCriteria, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 6);

    txtSearch = new wxTextCtrl(panel, wxID_ANY, "", wxDefaultPosition, wxSize(200, -1));
    row1->Add(txtSearch, 1, wxALIGN_CENTER_VERTICAL | wxRIGHT, 6);

    wxButton* btnSearch = new wxButton(panel, wxID_ANY, "2. Buscar Receta");
    row1->Add(btnSearch, 0, wxALIGN_CENTER_VERTICAL | wxRIGHT, 16);

    row1->Add(new wxStaticText(panel, wxID_ANY, "Filtrar Categoria:"), 0, wxALIGN_CENTER_VERTICAL |
wxRIGHT, 6);
    wxArrayString filterCats;
    filterCats.Add("Todas");
    filterCats.Add("Desayuno");
    filterCats.Add("Comida");
    filterCats.Add("Cena");
    filterCats.Add("Navideno");
    choiceFilterCategory = new wxChoice(panel, wxID_ANY, wxDefaultPosition, wxDefaultSize,
filterCats);
    choiceFilterCategory->SetSelection(0);
    row1->Add(choiceFilterCategory, 0, wxALIGN_CENTER_VERTICAL);

    wxBoxSizer* row2 = new wxBoxSizer(wxHORIZONTAL);
    wxButton* btnSortName = new wxButton(panel, wxID_ANY, "6a. Ordenar por Nombre");
    wxButton* btnSortTime = new wxButton(panel, wxID_ANY, "6b. Ordenar por Tiempo");
    wxButton* btnSaveDisk = new wxButton(panel, wxID_ANY, "8. Escribir al Disco");
    wxButton* btnLoadDisk = new wxButton(panel, wxID_ANY, "9. Leer del Disco");

    row2->Add(btnSortName, 1, wxEXPAND | wxRIGHT, 6);
    row2->Add(btnSortTime, 1, wxEXPAND | wxRIGHT, 6);
    row2->Add(btnSaveDisk, 1, wxEXPAND | wxRIGHT, 6);
    row2->Add(btnLoadDisk, 1, wxEXPAND);

    topBar->Add(row1, 0, wxEXPAND | wxALL, 4);
    topBar->Add(row2, 0, wxEXPAND | wxLEFT | wxRIGHT | wxBOTTOM, 4);

    pageSizer->Add(topBar, 0, wxEXPAND | wxALL, 6);

    // Zona central dividida: Izquierda (Tabla + Detalle) | Derecha (Formulario de
captura/modificacion)
    wxBoxSizer* centerSizer = new wxBoxSizer(wxHORIZONTAL);
```

```
    // Columna izquierda: Tabla de recetas y visor completo
    wxBoxSizer* leftCol = new wxBoxSizer(wxVERTICAL);
    recipeTable = new wxListCtrl(panel, wxID_ANY, wxDefaultPosition, wxSize(600, 260),
                                 wxLC_REPORT | wxLC_SINGLE_SEL | wxBORDER_SUNKEN);
    recipeTable->InsertColumn(0, "#", wxLIST_FORMAT_LEFT, 40);
    recipeTable->InsertColumn(1, "Nombre de la Receta", wxLIST_FORMAT_LEFT, 210);
    recipeTable->InsertColumn(2, "Autor (Name)", wxLIST_FORMAT_LEFT, 145);
    recipeTable->InsertColumn(3, "Tiempo (min)", wxLIST_FORMAT_LEFT, 95);
    recipeTable->InsertColumn(4, "Categoria", wxLIST_FORMAT_LEFT, 95);
    recipeTable->InsertColumn(5, "Ingredientes", wxLIST_FORMAT_LEFT, 85);

    leftCol->Add(recipeTable, 1, wxEXPAND | wxBOTTOM, 6);

    wxStaticBoxSizer* detailBox = new wxStaticBoxSizer(
        wxVERTICAL, panel, "7. Detalle Completo de la Receta Seleccionada (Ingredientes y
Procedimiento)");
    txtRecipeDetail = new wxTextCtrl(panel, wxID_ANY, "", wxDefaultPosition, wxSize(-1, 210),
                                     wxTE_MULTILINE | wxTE_READONLY);
    wxFont monoFont(10, wxFONTFAMILY_TELETYPE, wxFONTSTYLE_NORMAL, wxFONTWEIGHT_NORMAL);
    txtRecipeDetail->SetFont(monoFont);
    detailBox->Add(txtRecipeDetail, 1, wxEXPAND | wxALL, 4);
    leftCol->Add(detailBox, 1, wxEXPAND);

    centerSizer->Add(leftCol, 3, wxEXPAND | wxRIGHT, 8);

    // Columna derecha: Formulario de Receta y Botones CRUD
    wxStaticBoxSizer* rightBox = new wxStaticBoxSizer(wxVERTICAL, panel, "Datos de la Receta (Crear
/ Modificar)");
    wxFlexGridSizer* formGrid = new wxFlexGridSizer(6, 2, 8, 8);
    formGrid->AddGrowableCol(1, 1);

    formGrid->Add(new wxStaticText(panel, wxID_ANY, "Nombre del Platillo:"), 0,
wxALIGN_CENTER_VERTICAL);
    txtRecipeName = new wxTextCtrl(panel, wxID_ANY);
    formGrid->Add(txtRecipeName, 1, wxEXPAND);

    formGrid->Add(new wxStaticText(panel, wxID_ANY, "Elegir Chef Autor:"), 0,
wxALIGN_CENTER_VERTICAL);
    choiceChefSelector = new wxChoice(panel, wxID_ANY);
    formGrid->Add(choiceChefSelector, 1, wxEXPAND);

    formGrid->Add(new wxStaticText(panel, wxID_ANY, "Nombre Autor:"), 0, wxALIGN_CENTER_VERTICAL);
    txtAuthorFirst = new wxTextCtrl(panel, wxID_ANY);
    formGrid->Add(txtAuthorFirst, 1, wxEXPAND);

    formGrid->Add(new wxStaticText(panel, wxID_ANY, "Apellido Autor:"), 0,
wxALIGN_CENTER_VERTICAL);
    txtAuthorLast = new wxTextCtrl(panel, wxID_ANY);
    formGrid->Add(txtAuthorLast, 1, wxEXPAND);

    formGrid->Add(new wxStaticText(panel, wxID_ANY, "Tiempo Prep. (min):"), 0,
wxALIGN_CENTER_VERTICAL);
    txtPrepTime = new wxTextCtrl(panel, wxID_ANY, "30");
    formGrid->Add(txtPrepTime, 1, wxEXPAND);

    formGrid->Add(new wxStaticText(panel, wxID_ANY, "Categoria:"), 0, wxALIGN_CENTER_VERTICAL);
    wxArrayString catOptions;
    catOptions.Add("Desayuno");
    catOptions.Add("Comida");
    catOptions.Add("Cena");
    catOptions.Add("Navideno");
    choiceCategory = new wxChoice(panel, wxID_ANY, wxDefaultPosition, wxDefaultSize, catOptions);
```

```
    choiceCategory->SetSelection(1);
    formGrid->Add(choiceCategory, 1, wxEXPAND);

    rightBox->Add(formGrid, 0, wxEXPAND | wxALL, 6);

    rightBox->Add(new wxStaticText(panel, wxID_ANY, "Procedimiento de la Receta:"), 0, wxLEFT |
wxTOP, 6);
    txtProcedure = new wxTextCtrl(panel, wxID_ANY, "", wxDefaultPosition, wxSize(-1, 95),
wxTE_MULTILINE);
    rightBox->Add(txtProcedure, 1, wxEXPAND | wxALL, 6);

    wxButton* btnIngredients = new wxButton(panel, wxID_ANY, "Administrar Ingredientes de la
Receta...");
    rightBox->Add(btnIngredients, 0, wxEXPAND | wxLEFT | wxRIGHT | wxBOTTOM, 6);

    // Botones principales CRUD de Recetas
    wxGridSizer* crudGrid = new wxGridSizer(3, 2, 6, 6);
    wxButton* btnAdd = new wxButton(panel, wxID_ANY, "1. Agregar Receta");
    wxButton* btnModify = new wxButton(panel, wxID_ANY, "3. Modificar Receta");
    wxButton* btnDelete = new wxButton(panel, wxID_ANY, "4. Eliminar Receta");
    wxButton* btnDeleteAll = new wxButton(panel, wxID_ANY, "5. Eliminar Todas");
    wxButton* btnClearForm = new wxButton(panel, wxID_ANY, "Limpiar Formulario");
    wxButton* btnExit = new wxButton(panel, wxID_EXIT, "10. Salir");

    crudGrid->Add(btnAdd, 1, wxEXPAND);
    crudGrid->Add(btnModify, 1, wxEXPAND);
    crudGrid->Add(btnDelete, 1, wxEXPAND);
    crudGrid->Add(btnDeleteAll, 1, wxEXPAND);
    crudGrid->Add(btnClearForm, 1, wxEXPAND);
    crudGrid->Add(btnExit, 1, wxEXPAND);

    rightBox->Add(crudGrid, 0, wxEXPAND | wxALL, 6);
    centerSizer->Add(rightBox, 2, wxEXPAND);

    pageSizer->Add(centerSizer, 1, wxEXPAND | wxALL, 6);
    panel->SetSizer(pageSizer);

    // Conectar eventos
    recipeTable->Bind(wxEVT_LIST_ITEM_SELECTED, &RecipeUI::onSelectRecipe, this);
    btnAdd->Bind(wxEVT_BUTTON, &RecipeUI::onAddRecipe, this);
    btnModify->Bind(wxEVT_BUTTON, &RecipeUI::onModifyRecipe, this);
    btnDelete->Bind(wxEVT_BUTTON, &RecipeUI::onDeleteRecipe, this);
    btnDeleteAll->Bind(wxEVT_BUTTON, &RecipeUI::onDeleteAllRecipes, this);
    btnIngredients->Bind(wxEVT_BUTTON, &RecipeUI::onManageIngredients, this);
    btnSearch->Bind(wxEVT_BUTTON, &RecipeUI::onFindRecipe, this);
    choiceFilterCategory->Bind(wxEVT_CHOICE, &RecipeUI::onFilterCategory, this);
    btnSortName->Bind(wxEVT_BUTTON, &RecipeUI::onSortByName, this);
    btnSortTime->Bind(wxEVT_BUTTON, &RecipeUI::onSortByPrepTime, this);
    btnSaveDisk->Bind(wxEVT_BUTTON, &RecipeUI::onWriteToDisk, this);
    btnLoadDisk->Bind(wxEVT_BUTTON, &RecipeUI::onReadFromDisk, this);
    choiceChefSelector->Bind(wxEVT_CHOICE, &RecipeUI::onChefSelectedForRecipe, this);
    btnClearForm->Bind(wxEVT_BUTTON, &RecipeUI::onClearRecipeForm, this);
    btnExit->Bind(wxEVT_BUTTON, [this](wxCommandEvent&) { Close(true); });

    return panel;
}

wxPanel* RecipeUI::createChefsPage(wxWindow* parent) {
    wxPanel* panel = new wxPanel(parent, wxID_ANY);
    wxBoxSizer* mainSizer = new wxBoxSizer(wxVERTICAL);

    wxStaticText* lblInfo = new wxStaticText(
```

```
        panel, wxID_ANY,
        wxString::FromUTF8("Gestion de Autores / Chefs — Demostracion de Herencia (Chef : public
Persona), "
                           "Realizacion (IMostrable), Composicion (Name) y Polimorfismo"));
    wxFont boldFont = lblInfo->GetFont();
    boldFont.SetWeight(wxFONTWEIGHT_BOLD);
    lblInfo->SetFont(boldFont);
    mainSizer->Add(lblInfo, 0, wxALL, 8);

    wxBoxSizer* contentSizer = new wxBoxSizer(wxHORIZONTAL);

    // Tabla de Chefs
    chefTable = new wxListCtrl(panel, wxID_ANY, wxDefaultPosition, wxSize(620, 320),
                               wxLC_REPORT | wxLC_SINGLE_SEL | wxBORDER_SUNKEN);
    chefTable->InsertColumn(0, "#", wxLIST_FORMAT_LEFT, 40);
    chefTable->InsertColumn(1, "Nombre (Name)", wxLIST_FORMAT_LEFT, 150);
    chefTable->InsertColumn(2, "Edad (Persona)", wxLIST_FORMAT_LEFT, 95);
    chefTable->InsertColumn(3, "Especialidad (Chef)", wxLIST_FORMAT_LEFT, 185);
    chefTable->InsertColumn(4, "Experiencia", wxLIST_FORMAT_LEFT, 95);
    chefTable->InsertColumn(5, "Rol Polimorfico", wxLIST_FORMAT_LEFT, 110);

    contentSizer->Add(chefTable, 3, wxEXPAND | wxRIGHT, 8);

    // Formulario de Chef
    wxStaticBoxSizer* formBox = new wxStaticBoxSizer(wxVERTICAL, panel, "Instancia de Clase Chef
(Hereda de Persona)");
    wxFlexGridSizer* grid = new wxFlexGridSizer(5, 2, 8, 8);
    grid->AddGrowableCol(1, 1);

    grid->Add(new wxStaticText(panel, wxID_ANY, "Nombre (Name.first):"), 0,
wxALIGN_CENTER_VERTICAL);
    txtChefFirst = new wxTextCtrl(panel, wxID_ANY);
    grid->Add(txtChefFirst, 1, wxEXPAND);

    grid->Add(new wxStaticText(panel, wxID_ANY, "Apellido (Name.last):"), 0,
wxALIGN_CENTER_VERTICAL);
    txtChefLast = new wxTextCtrl(panel, wxID_ANY);
    grid->Add(txtChefLast, 1, wxEXPAND);

    grid->Add(new wxStaticText(panel, wxID_ANY, "Edad (Persona.edad):"), 0,
wxALIGN_CENTER_VERTICAL);
    txtChefAge = new wxTextCtrl(panel, wxID_ANY, "30");
    grid->Add(txtChefAge, 1, wxEXPAND);

    grid->Add(new wxStaticText(panel, wxID_ANY, "Especialidad (Chef):"), 0,
wxALIGN_CENTER_VERTICAL);
    txtChefSpecialty = new wxTextCtrl(panel, wxID_ANY, "Cocina Mexicana");
    grid->Add(txtChefSpecialty, 1, wxEXPAND);

    grid->Add(new wxStaticText(panel, wxID_ANY, "Anios Experiencia:"), 0, wxALIGN_CENTER_VERTICAL);
    txtChefExp = new wxTextCtrl(panel, wxID_ANY, "5");
    grid->Add(txtChefExp, 1, wxEXPAND);

    formBox->Add(grid, 0, wxEXPAND | wxALL, 8);

    wxBoxSizer* btnRow = new wxBoxSizer(wxHORIZONTAL);
    wxButton* btnAddChef = new wxButton(panel, wxID_ANY, "Crear Chef");
    wxButton* btnModChef = new wxButton(panel, wxID_ANY, "Modificar Chef");
    wxButton* btnDelChef = new wxButton(panel, wxID_ANY, "Eliminar Chef");
    btnRow->Add(btnAddChef, 1, wxRIGHT, 4);
    btnRow->Add(btnModChef, 1, wxRIGHT, 4);
    btnRow->Add(btnDelChef, 1);
```

```
    formBox->Add(btnRow, 0, wxEXPAND | wxALL, 6);

    wxButton* btnPoly = new wxButton(panel, wxID_ANY, "Demostrar Polimorfismo (Persona vs Chef)");
    formBox->Add(btnPoly, 0, wxEXPAND | wxALL, 6);

    contentSizer->Add(formBox, 2, wxEXPAND);
    mainSizer->Add(contentSizer, 1, wxEXPAND | wxALL, 8);

    // Consola de salida polimorfica
    wxStaticBoxSizer* polyBox = new wxStaticBoxSizer(
        wxVERTICAL, panel, "Salida del Metodo Virtual obtenerFicha() /
consultarAutorPolimorfico(const Persona&)");
    txtPolymorphismOutput = new wxTextCtrl(panel, wxID_ANY, "", wxDefaultPosition, wxSize(-1, 130),
                                           wxTE_MULTILINE | wxTE_READONLY);
    wxFont monoFont(10, wxFONTFAMILY_TELETYPE, wxFONTSTYLE_NORMAL, wxFONTWEIGHT_NORMAL);
    txtPolymorphismOutput->SetFont(monoFont);
    polyBox->Add(txtPolymorphismOutput, 1, wxEXPAND | wxALL, 4);
    mainSizer->Add(polyBox, 0, wxEXPAND | wxLEFT | wxRIGHT | wxBOTTOM, 8);

    panel->SetSizer(mainSizer);

    // Eventos de Chefs
    chefTable->Bind(wxEVT_LIST_ITEM_SELECTED, &RecipeUI::onSelectChef, this);
    btnAddChef->Bind(wxEVT_BUTTON, &RecipeUI::onAddChef, this);
    btnModChef->Bind(wxEVT_BUTTON, &RecipeUI::onModifyChef, this);
    btnDelChef->Bind(wxEVT_BUTTON, &RecipeUI::onDeleteChef, this);
    btnPoly->Bind(wxEVT_BUTTON, &RecipeUI::onDemonstratePolymorphism, this);

    return panel;
}

void RecipeUI::refreshRecipeTable(const std::string& filterCategory) {
    recipeTable->DeleteAllItems();
    displayedIndices.clear();

    long row = 0;
    for (size_t i = 0; i < this->recipeListRef.size(); i++) {
        const Recipe& rec = this->recipeListRef[i];
        if (filterCategory != "Todas" && !filterCategory.empty() && rec.getCategory() !=
filterCategory) {
            continue;
        }

        displayedIndices.push_back(static_cast<int>(i));
        long idx = recipeTable->InsertItem(row, to_string(i));
        recipeTable->SetItem(idx, 1, wxString::FromUTF8(rec.getName()));
        recipeTable->SetItem(idx, 2, wxString::FromUTF8(rec.getAuthorName().toString()));
        recipeTable->SetItem(idx, 3, to_string(rec.getPrepTime()) + " min");
        recipeTable->SetItem(idx, 4, wxString::FromUTF8(rec.getCategory()));
        recipeTable->SetItem(idx, 5, to_string(rec.getIngredientList().size()));
        row++;
    }
    recipeTable->Refresh();
    recipeTable->Update();

    SetStatusText(wxString::FromUTF8("Total de recetas en el vector: " + to_string(this-
>recipeListRef.size()) +
                                     " | Mostradas: " + to_string(displayedIndices.size())));
}

void RecipeUI::refreshChefTable() {
    chefTable->DeleteAllItems();
```

```
    stringstream polySS;
    polySS << "=== LISTADO POLIMORFICO DE INSTANCIAS (consultarAutorPolimorfico) ===\n";

    for (size_t i = 0; i < chefList.size(); i++) {
        const Chef& c = chefList[i];
        long idx = chefTable->InsertItem(i, to_string(i));
        chefTable->SetItem(idx, 1, wxString::FromUTF8(c.getNombre().toString()));
        chefTable->SetItem(idx, 2, to_string(c.getEdad()));
        chefTable->SetItem(idx, 3, wxString::FromUTF8(c.getEspecialidad()));
        chefTable->SetItem(idx, 4, to_string(c.getAniosExperiencia()) + " anios");
        chefTable->SetItem(idx, 5, wxString::FromUTF8(c.obtenerRol()));

        polySS << "[" << i << "] " << consultarAutorPolimorfico(c) << "\n";
    }
    chefTable->Refresh();
    chefTable->Update();
    txtPolymorphismOutput->SetValue(wxString::FromUTF8(polySS.str()));
}

void RecipeUI::updateChefChoiceSelector() {
    choiceChefSelector->Clear();
    choiceChefSelector->Append("-- Escribir autor manualmente --");
    for (size_t i = 0; i < chefList.size(); i++) {
        choiceChefSelector->Append(wxString::FromUTF8(
            chefList[i].getNombre().toString() + " (" + chefList[i].getEspecialidad() + ")"));
    }
    choiceChefSelector->SetSelection(0);
}

void RecipeUI::showRecipeDetails(int realIndex) {
    if (realIndex < 0 || realIndex >= static_cast<int>(this->recipeListRef.size())) {
        txtRecipeDetail->Clear();
        return;
    }

    const Recipe& rec = this->recipeListRef[realIndex];
    stringstream ss;
    ss << "================================================================================\n";
    ss << " RECETA: " << rec.getName() << "\n";
    ss << " AUTOR : " << rec.getAuthorName().toString()
       << "   |   TIEMPO: " << rec.getPrepTime() << " minutos"
       << "   |   CATEGORIA: " << rec.getCategory() << "\n";
    ss << "================================================================================\n\n";
    ss << " | " << left << setw(wIngredientName) << "INGREDIENTE"
       << " | " << left << setw(wQuantity) << "CANTIDAD"
       << " | " << left << setw(wUnit) << "UNIDAD" << " |\n";
    ss << " ------------------------------------------------------------------------------\n";

    if (rec.getIngredientList().empty()) {
        ss << "   (Sin ingredientes registrados)\n";
    } else {
        for (size_t i = 0; i < rec.getIngredientList().size(); i++) {
            ss << rec.getIngredientList()[i].toString();
        }
    }

    ss << "\nPROCEDIMIENTO:\n" << rec.getProcedure() << "\n";
    txtRecipeDetail->SetValue(wxString::FromUTF8(ss.str()));
}

void RecipeUI::onSelectRecipe(wxListEvent& event) {
    long tableRow = event.GetIndex();
```

```
    if (tableRow < 0 || tableRow >= static_cast<long>(displayedIndices.size())) {
        return;
    }

    selectedRecipePos = displayedIndices[tableRow];
    const Recipe& rec = this->recipeListRef[selectedRecipePos];

    // Llenar el formulario con los datos de la receta seleccionada
    txtRecipeName->SetValue(wxString::FromUTF8(rec.getName()));
    txtAuthorFirst->SetValue(wxString::FromUTF8(rec.getAuthorName().getFirst()));
    txtAuthorLast->SetValue(wxString::FromUTF8(rec.getAuthorName().getLast()));
    txtPrepTime->SetValue(to_string(rec.getPrepTime()));
    txtProcedure->SetValue(wxString::FromUTF8(rec.getProcedure()));

    int catIdx = choiceCategory->FindString(wxString::FromUTF8(rec.getCategory()));
    if (catIdx != wxNOT_FOUND) {
        choiceCategory->SetSelection(catIdx);
    }

    tempIngredients = rec.getIngredientList();
    showRecipeDetails(selectedRecipePos);
}

void RecipeUI::onChefSelectedForRecipe(wxCommandEvent&) {
    int sel = choiceChefSelector->GetSelection();
    if (sel > 0 && (sel - 1) < static_cast<int>(chefList.size())) {
        const Chef& chosenChef = chefList[sel - 1];
        txtAuthorFirst->SetValue(wxString::FromUTF8(chosenChef.getNombre().getFirst()));
        txtAuthorLast->SetValue(wxString::FromUTF8(chosenChef.getNombre().getLast()));
    }
}

void RecipeUI::onClearRecipeForm(wxCommandEvent&) {
    selectedRecipePos = -1;
    txtRecipeName->Clear();
    txtAuthorFirst->Clear();
    txtAuthorLast->Clear();
    choiceChefSelector->SetSelection(0);
    txtPrepTime->SetValue("30");
    choiceCategory->SetSelection(1);
    txtProcedure->Clear();
    tempIngredients.clear();
    txtRecipeDetail->Clear();
}

void RecipeUI::onAddRecipe(wxCommandEvent&) {
    string rName = txtRecipeName->GetValue().ToUTF8().data();
    string aFirst = txtAuthorFirst->GetValue().ToUTF8().data();
    string aLast = txtAuthorLast->GetValue().ToUTF8().data();
    string timeStr = txtPrepTime->GetValue().ToUTF8().data();
    string proc = txtProcedure->GetValue().ToUTF8().data();
    string cat = choiceCategory->GetStringSelection().ToUTF8().data();

    if (rName.empty() || aFirst.empty() || aLast.empty() || proc.empty()) {
        wxMessageBox("Por favor completa el nombre de la receta, autor y procedimiento.",
                     "Datos incompletos", wxOK | wxICON_WARNING, this);
        return;
    }

    int prepMinutes = 0;
    try {
        prepMinutes = stoi(timeStr);
```

```
    } catch (...) {
        wxMessageBox("El tiempo de preparacion debe ser un numero en minutos.",
                     "Error en tiempo", wxOK | wxICON_ERROR, this);
        return;
    }

    Recipe myRecipe;
    myRecipe.setName(rName);

    int chefSel = choiceChefSelector->GetSelection();
    if (chefSel > 0 && (chefSel - 1) < static_cast<int>(chefList.size())) {
        // Relacion de colaboracion entre Recipe y Chef
        myRecipe.asignarChefAutor(chefList[chefSel - 1]);
    } else {
        Name tempName(aFirst, aLast);
        myRecipe.setAuthor(tempName);
    }

    myRecipe.setPrepTime(prepMinutes);
    myRecipe.setProcedure(proc);
    myRecipe.setCategory(cat);
    myRecipe.getIngredientList() = tempIngredients;

    int askIng = wxMessageBox("¿Deseas abrir el administrador de ingredientes antes de guardar la
receta?",
                              "Ingredientes de la receta", wxYES_NO | wxICON_QUESTION, this);
    if (askIng == wxYES) {
        IngredientUI ingDialog(this, myRecipe.getIngredientList(), myRecipe.getName());
        ingDialog.manageIngredient();
    }

    this->recipeListRef.push_back(myRecipe);
    selectedRecipePos = static_cast<int>(this->recipeListRef.size()) - 1;
    tempIngredients = myRecipe.getIngredientList();

    refreshRecipeTable(choiceFilterCategory->GetStringSelection().ToUTF8().data());
    showRecipeDetails(selectedRecipePos);
    wxMessageBox("Receta agregada con exito al vector!", "Receta Agregada", wxOK |
wxICON_INFORMATION, this);
}

void RecipeUI::onModifyRecipe(wxCommandEvent&) {
    if (selectedRecipePos < 0 || selectedRecipePos >= static_cast<int>(this->recipeListRef.size()))
{
        // Intentar buscar por el nombre escrito en el formulario
        string targetName = txtRecipeName->GetValue().ToUTF8().data();
        for (size_t i = 0; i < this->recipeListRef.size(); i++) {
            if (this->recipeListRef[i].getName() == targetName) {
                selectedRecipePos = static_cast<int>(i);
                break;
            }
        }
    }

    if (selectedRecipePos < 0 || selectedRecipePos >= static_cast<int>(this->recipeListRef.size()))
{
        wxMessageBox("Selecciona primero una receta de la tabla para modificarla.",
                     "Receta no seleccionada", wxOK | wxICON_WARNING, this);
        return;
    }

    string rName = txtRecipeName->GetValue().ToUTF8().data();
```

```
    string aFirst = txtAuthorFirst->GetValue().ToUTF8().data();
    string aLast = txtAuthorLast->GetValue().ToUTF8().data();
    string timeStr = txtPrepTime->GetValue().ToUTF8().data();
    string proc = txtProcedure->GetValue().ToUTF8().data();
    string cat = choiceCategory->GetStringSelection().ToUTF8().data();

    int prepMinutes = this->recipeListRef[selectedRecipePos].getPrepTime();
    try {
        prepMinutes = stoi(timeStr);
    } catch (...) { }

    Recipe& target = this->recipeListRef[selectedRecipePos];
    if (!rName.empty()) target.setName(rName);
    if (!aFirst.empty() || !aLast.empty()) target.setAuthor(Name(aFirst, aLast));
    target.setPrepTime(prepMinutes);
    if (!proc.empty()) target.setProcedure(proc);
    if (!cat.empty()) target.setCategory(cat);

    refreshRecipeTable(choiceFilterCategory->GetStringSelection().ToUTF8().data());
    showRecipeDetails(selectedRecipePos);
    wxMessageBox("Receta modificada correctamente!", "Receta Actualizada", wxOK |
wxICON_INFORMATION, this);
}

void RecipeUI::onManageIngredients(wxCommandEvent&) {
    if (selectedRecipePos >= 0 && selectedRecipePos < static_cast<int>(this->recipeListRef.size()))
{
        Recipe& rec = this->recipeListRef[selectedRecipePos];
        IngredientUI ingDialog(this, rec.getIngredientList(), rec.getName());
        ingDialog.manageIngredient();
        tempIngredients = rec.getIngredientList();
        refreshRecipeTable(choiceFilterCategory->GetStringSelection().ToUTF8().data());
        showRecipeDetails(selectedRecipePos);
    } else {
        IngredientUI ingDialog(this, tempIngredients, "Nueva Receta");
        ingDialog.manageIngredient();
    }
}

void RecipeUI::onDeleteRecipe(wxCommandEvent&) {
    if (selectedRecipePos < 0 || selectedRecipePos >= static_cast<int>(this->recipeListRef.size()))
{
        string targetName = txtRecipeName->GetValue().ToUTF8().data();
        for (size_t i = 0; i < this->recipeListRef.size(); i++) {
            if (this->recipeListRef[i].getName() == targetName) {
                selectedRecipePos = static_cast<int>(i);
                break;
            }
        }
    }

    if (selectedRecipePos < 0 || selectedRecipePos >= static_cast<int>(this->recipeListRef.size()))
{
        wxMessageBox("Selecciona una receta de la tabla o escribe su nombre para eliminarla.",
                     "Receta no encontrada", wxOK | wxICON_WARNING, this);
        return;
    }

    string nameToDelete = this->recipeListRef[selectedRecipePos].getName();
    int resp = wxMessageBox(
        wxString::FromUTF8("¿Estas seguro de que deseas eliminar la receta '" + nameToDelete +
"'?"),
```

```
        "Confirmar eliminacion", wxYES_NO | wxICON_QUESTION, this);

    if (resp == wxYES) {
        this->recipeListRef.erase(this->recipeListRef.begin() + selectedRecipePos);
        selectedRecipePos = -1;
        txtRecipeDetail->Clear();
        refreshRecipeTable(choiceFilterCategory->GetStringSelection().ToUTF8().data());
        wxMessageBox("Receta eliminada correctamente.", "Eliminada", wxOK | wxICON_INFORMATION,
this);
    }
}

void RecipeUI::onDeleteAllRecipes(wxCommandEvent&) {
    if (this->recipeListRef.empty()) {
        wxMessageBox("El recetario ya se encuentra vacio.", "Aviso", wxOK | wxICON_INFORMATION,
this);
        return;
    }

    int resp = wxMessageBox("¿Estas seguro de eliminar TODAS LAS RECETAS del vector?",
                            "Eliminar todas las recetas", wxYES_NO | wxICON_WARNING, this);
    if (resp == wxYES) {
        this->recipeListRef.clear();
        selectedRecipePos = -1;
        txtRecipeDetail->Clear();
        refreshRecipeTable();
        wxMessageBox("TODAS LAS RECETAS FUERON ELIMINADAS.", "Recetario Vacio", wxOK |
wxICON_INFORMATION, this);
    }
}

void RecipeUI::onFindRecipe(wxCommandEvent&) {
    string query = txtSearch->GetValue().ToUTF8().data();
    if (query.empty()) {
        wxMessageBox("Escribe el nombre del platillo o del autor a buscar.",
                     "Busqueda vacia", wxOK | wxICON_WARNING, this);
        return;
    }

    int criteria = choiceSearchCriteria->GetSelection(); // 0 = Nombre platillo, 1 = Autor
    int foundPos = -1;

    for (size_t i = 0; i < this->recipeListRef.size(); i++) {
        if (criteria == 0) {
            if (this->recipeListRef[i].getName().find(query) != string::npos) {
                foundPos = static_cast<int>(i);
                break;
            }
        } else {
            if (this->recipeListRef[i].getAuthorName().toString().find(query) != string::npos) {
                foundPos = static_cast<int>(i);
                break;
            }
        }
    }

    if (foundPos == -1) {
        wxMessageBox("La receta no fue encontrada...", "Sin resultados", wxOK | wxICON_INFORMATION,
this);
    } else {
        selectedRecipePos = foundPos;
        choiceFilterCategory->SetSelection(0);
```

```
        refreshRecipeTable("Todas");

        recipeTable->SetItemState(foundPos, wxLIST_STATE_SELECTED | wxLIST_STATE_FOCUSED,
                                  wxLIST_STATE_SELECTED | wxLIST_STATE_FOCUSED);
        recipeTable->EnsureVisible(foundPos);
        showRecipeDetails(foundPos);
    }
}

void RecipeUI::onFilterCategory(wxCommandEvent&) {
    string cat = choiceFilterCategory->GetStringSelection().ToUTF8().data();
    refreshRecipeTable(cat);
}

void RecipeUI::onSortByName(wxCommandEvent&) {
    std::sort(this->recipeListRef.begin(), this->recipeListRef.end(),
              [](const Recipe& a, const Recipe& b) {
                  return Recipe::compareByName(a, b) < 0;
              });
    refreshRecipeTable(choiceFilterCategory->GetStringSelection().ToUTF8().data());
    SetStatusText("La lista fue ordenada por nombre de la receta!");
}

void RecipeUI::onSortByPrepTime(wxCommandEvent&) {
    std::sort(this->recipeListRef.begin(), this->recipeListRef.end(),
              [](const Recipe& a, const Recipe& b) {
                  return Recipe::compareByPrepTime(a, b) < 0;
              });
    refreshRecipeTable(choiceFilterCategory->GetStringSelection().ToUTF8().data());
    SetStatusText("La lista fue ordenada por tiempo de preparacion!");
}

void RecipeUI::onWriteToDisk(wxCommandEvent&) {
    wxFileDialog saveFileDialog(this, "Guardar recetario en disco", "", "recetas.txt",
                                "Archivos de texto (*.txt)|*.txt", wxFD_SAVE |
wxFD_OVERWRITE_PROMPT);
    if (saveFileDialog.ShowModal() == wxID_CANCEL) {
        return;
    }

    string path = saveFileDialog.GetPath().ToUTF8().data();
    if (saveToFile(path)) {
        wxMessageBox("El recetario se guardo correctamente en:\n" + path,
                     "Guardado exitoso", wxOK | wxICON_INFORMATION, this);
    } else {
        wxMessageBox("Fue imposible abrir el archivo para escritura.",
                     "Error", wxOK | wxICON_ERROR, this);
    }
}

void RecipeUI::onReadFromDisk(wxCommandEvent&) {
    wxFileDialog openFileDialog(this, "Leer recetario del disco", "", "recetas.txt",
                                "Archivos de texto (*.txt)|*.txt", wxFD_OPEN |
wxFD_FILE_MUST_EXIST);
    if (openFileDialog.ShowModal() == wxID_CANCEL) {
        return;
    }

    string path = openFileDialog.GetPath().ToUTF8().data();
    loadFromFile(path);
    wxMessageBox("Recetario recuperado! Total de recetas: " + to_string(this-
>recipeListRef.size()),
```

```
                "Lectura exitosa", wxOK | wxICON_INFORMATION, this);
}

void RecipeUI::loadFromFile(const std::string& filename) {
    ifstream myFile(filename);
    if (!myFile.is_open()) {
        return;
    }

    string content;
    getline(myFile, content);
    myFile.close();

    this->recipeListRef.clear();
    stringstream ss(content);
    string token;
    while (getline(ss, token, '|')) {
        if (token.empty()) {
            continue;
        }
        Recipe myRecipe;
        myRecipe.deserialize(token, '*');
        this->recipeListRef.push_back(myRecipe);
    }

    refreshRecipeTable(choiceFilterCategory->GetStringSelection().ToUTF8().data());
    if (!this->recipeListRef.empty()) {
        selectedRecipePos = 0;
        const Recipe& rec = this->recipeListRef[0];
        txtRecipeName->SetValue(wxString::FromUTF8(rec.getName()));
        txtAuthorFirst->SetValue(wxString::FromUTF8(rec.getAuthorName().getFirst()));
        txtAuthorLast->SetValue(wxString::FromUTF8(rec.getAuthorName().getLast()));
        txtPrepTime->SetValue(to_string(rec.getPrepTime()));
        txtProcedure->SetValue(wxString::FromUTF8(rec.getProcedure()));
        int catIdx = choiceCategory->FindString(wxString::FromUTF8(rec.getCategory()));
        if (catIdx != wxNOT_FOUND) choiceCategory->SetSelection(catIdx);
        tempIngredients = rec.getIngredientList();
        recipeTable->SetItemState(0, wxLIST_STATE_SELECTED | wxLIST_STATE_FOCUSED,
                                  wxLIST_STATE_SELECTED | wxLIST_STATE_FOCUSED);
        showRecipeDetails(0);
    }
}

bool RecipeUI::saveToFile(const std::string& filename) {
    ofstream myFile(filename, ios_base::trunc);
    if (!myFile.is_open()) {
        return false;
    }

    string serialized;
    for (size_t i = 0; i < this->recipeListRef.size(); i++) {
        serialized += this->recipeListRef[i].serialize('*') + '|';
    }
    myFile << serialized;
    myFile.close();
    return true;
}

// ==================== Eventos de la Pestaña de Chefs ====================

void RecipeUI::onSelectChef(wxListEvent& event) {
    selectedChefPos = event.GetIndex();
```

```
    if (selectedChefPos >= 0 && selectedChefPos < static_cast<int>(chefList.size())) {
        const Chef& c = chefList[selectedChefPos];
        txtChefFirst->SetValue(wxString::FromUTF8(c.getNombre().getFirst()));
        txtChefLast->SetValue(wxString::FromUTF8(c.getNombre().getLast()));
        txtChefAge->SetValue(to_string(c.getEdad()));
        txtChefSpecialty->SetValue(wxString::FromUTF8(c.getEspecialidad()));
        txtChefExp->SetValue(to_string(c.getAniosExperiencia()));
    }
}

void RecipeUI::onAddChef(wxCommandEvent&) {
    string first = txtChefFirst->GetValue().ToUTF8().data();
    string last = txtChefLast->GetValue().ToUTF8().data();
    string ageStr = txtChefAge->GetValue().ToUTF8().data();
    string spec = txtChefSpecialty->GetValue().ToUTF8().data();
    string expStr = txtChefExp->GetValue().ToUTF8().data();

    if (first.empty() || last.empty() || spec.empty()) {
        wxMessageBox("Completa nombre, apellido y especialidad del Chef.",
                     "Campos requeridos", wxOK | wxICON_WARNING, this);
        return;
    }

    int age = 25, exp = 1;
    try { age = stoi(ageStr); } catch (...) { }
    try { exp = stoi(expStr); } catch (...) { }

    Chef newChef(Name(first, last), age, spec, exp);
    chefList.push_back(newChef);

    refreshChefTable();
    updateChefChoiceSelector();
    wxMessageBox("Instancia de Chef creada y agregada al vector!", "Chef Creado", wxOK |
wxICON_INFORMATION, this);
}

void RecipeUI::onModifyChef(wxCommandEvent&) {
    if (selectedChefPos < 0 || selectedChefPos >= static_cast<int>(chefList.size())) {
        wxMessageBox("Selecciona un Chef de la tabla para modificarlo.",
                     "Sin seleccion", wxOK | wxICON_WARNING, this);
        return;
    }

    string first = txtChefFirst->GetValue().ToUTF8().data();
    string last = txtChefLast->GetValue().ToUTF8().data();
    string ageStr = txtChefAge->GetValue().ToUTF8().data();
    string spec = txtChefSpecialty->GetValue().ToUTF8().data();
    string expStr = txtChefExp->GetValue().ToUTF8().data();

    int age = chefList[selectedChefPos].getEdad();
    int exp = chefList[selectedChefPos].getAniosExperiencia();
    try { age = stoi(ageStr); } catch (...) { }
    try { exp = stoi(expStr); } catch (...) { }

    chefList[selectedChefPos].setNombre(Name(first, last));
    chefList[selectedChefPos].setEdad(age);
    chefList[selectedChefPos].setEspecialidad(spec);
    chefList[selectedChefPos].setAniosExperiencia(exp);

    refreshChefTable();
    updateChefChoiceSelector();
    wxMessageBox("Datos del Chef modificados con exito!", "Chef Modificado", wxOK |
```

```
wxICON_INFORMATION, this);
}

void RecipeUI::onDeleteChef(wxCommandEvent&) {
    if (selectedChefPos < 0 || selectedChefPos >= static_cast<int>(chefList.size())) {
        wxMessageBox("Selecciona un Chef de la tabla para eliminarlo.",
                     "Sin seleccion", wxOK | wxICON_WARNING, this);
        return;
    }

    string fullName = chefList[selectedChefPos].getNombre().toString();
    int resp = wxMessageBox(
        wxString::FromUTF8("¿Deseas eliminar al Chef '" + fullName + "'?"),
        "Confirmar eliminacion", wxYES_NO | wxICON_QUESTION, this);

    if (resp == wxYES) {
        chefList.erase(chefList.begin() + selectedChefPos);
        selectedChefPos = -1;
        refreshChefTable();
        updateChefChoiceSelector();
    }
}

void RecipeUI::onDemonstratePolymorphism(wxCommandEvent&) {
    Persona personaGenerica(Name("Diego", "Ruelas"), 20);
    Chef chefEspecializado(Name("Ana", "Garcia"), 34, "Cocina Navidena", 12);

    // Arreglo de punteros a la clase base Persona para demostrar polimorfismo en tiempo de
ejecucion
    vector<const Persona*> listaPolimorfica;
    listaPolimorfica.push_back(&personaGenerica);
    listaPolimorfica.push_back(&chefEspecializado);

    stringstream ss;
    ss << "=== DEMOSTRACION DE LOS 4 PILARES DE LA POO (POLIMORFISMO Y HERENCIA) ===\n\n";
    ss << "Invocando consultarAutorPolimorfico(const Persona&) con enlace dinamico (virtual /
override):\n\n";
    for (size_t i = 0; i < listaPolimorfica.size(); i++) {
        ss << "  Objeto " << i + 1 << " -> " << consultarAutorPolimorfico(*listaPolimorfica[i]) <<
"\n";
    }

    txtPolymorphismOutput->SetValue(wxString::FromUTF8(ss.str()));
    wxMessageBox(wxString::FromUTF8(ss.str()), "Demostracion de Polimorfismo", wxOK |
wxICON_INFORMATION, this);
}
```

**Archivo: `src/main.cpp` — Punto de entrada wxWidgets (RecetarioApp : public wxApp)**

```
#include <wx/wx.h>
#include <vector>
#include "recipe.hpp"
#include "recipeui.hpp"

using namespace std;

class RecetarioApp : public wxApp {
private:
    std::vector<Recipe> miListaDeRecetas;
```

```
public:
    virtual bool OnInit() override {
        RecipeUI* ui = new RecipeUI(miListaDeRecetas);
        ui->Show(true);
        return true;
    }
};

wxIMPLEMENT_APP(RecetarioApp);
```

## 5. Pruebas de Funcionamiento y Diagrama de Objetos

### 5.1 Diagrama de Objetos UML (Instancias en Tiempo de Ejecución)

Para verificar el comportamiento de las instancias en memoria durante la ejecución del programa, la **Figura 3** ilustra el **Diagrama de Objetos UML** correspondiente a una receta real cargada en el sistema (`receta1 : Recipe` con nombre "*Pavo Relleno Suprema*"), su objeto compuesto `autor1 : Name`, su primer ingrediente dentro del vector (`ing1 : Ingredient`) y el objeto derivado `chef1 : Chef` asignado como autor:

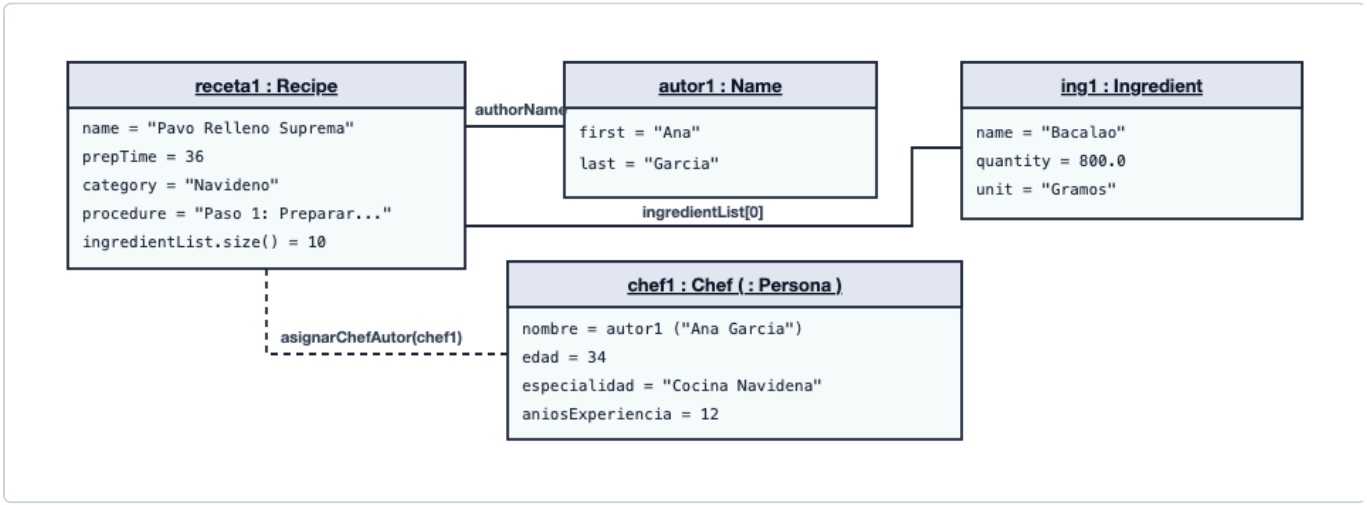



*Figura 3. Diagrama de Objetos UML mostrando el estado concreto de las instancias y sus enlaces en ejecución.*

### 5.2 Evidencia de Ejecución de la Interfaz Gráfica (wxWidgets)

A continuación se presentan las capturas de pantalla reales del sistema compilado y en ejecución sobre macOS utilizando **wxWidgets 3.3**, validando cada uno de los casos de uso definidos en el análisis:

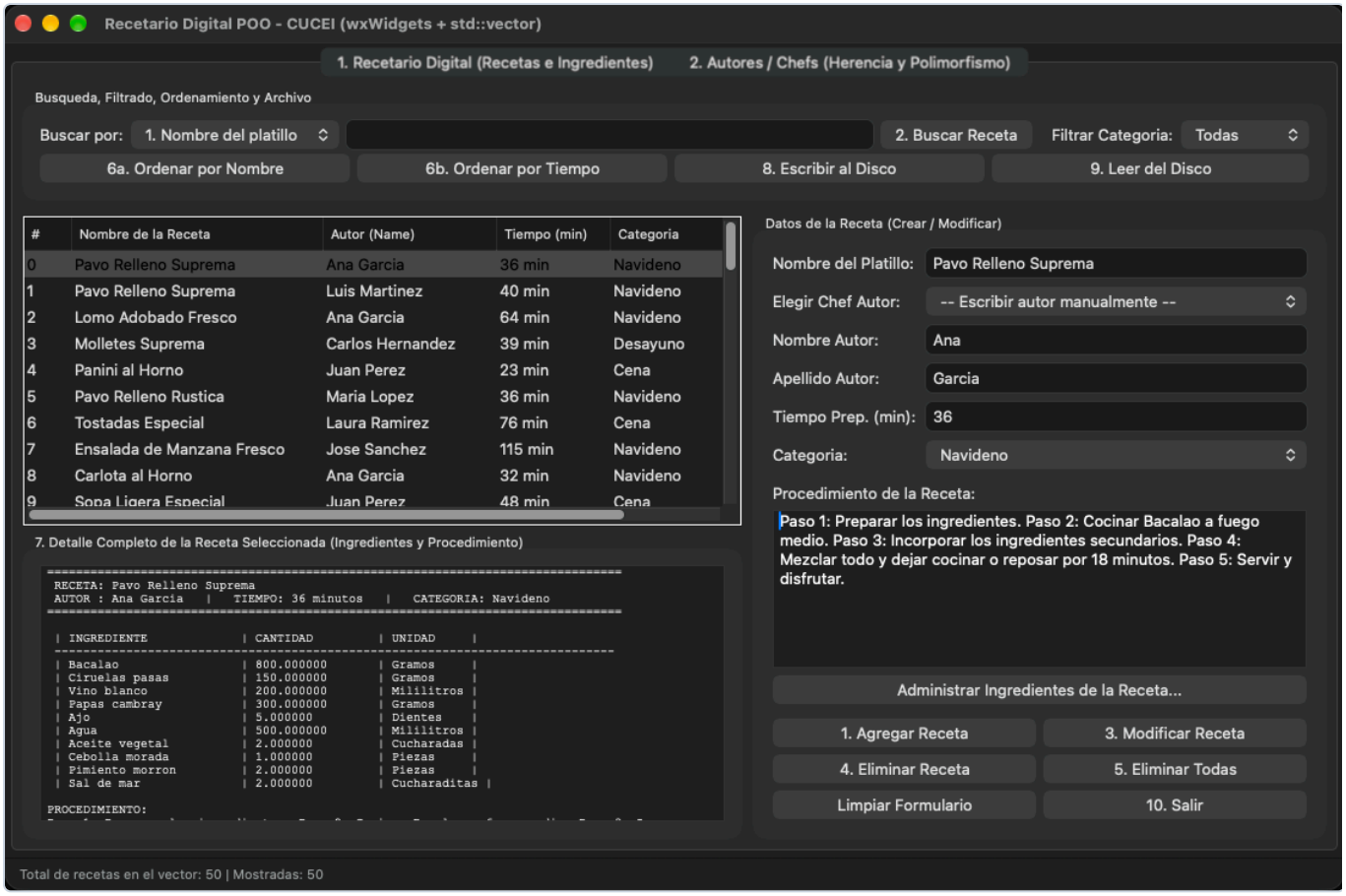



*Figura 4. Ventana principal (RecipeUI : public wxFrame) con las 50 recetas cargadas automáticamente desde recetas.txt en un std::vector<Recipe> y visualización detallada de la receta seleccionada.*

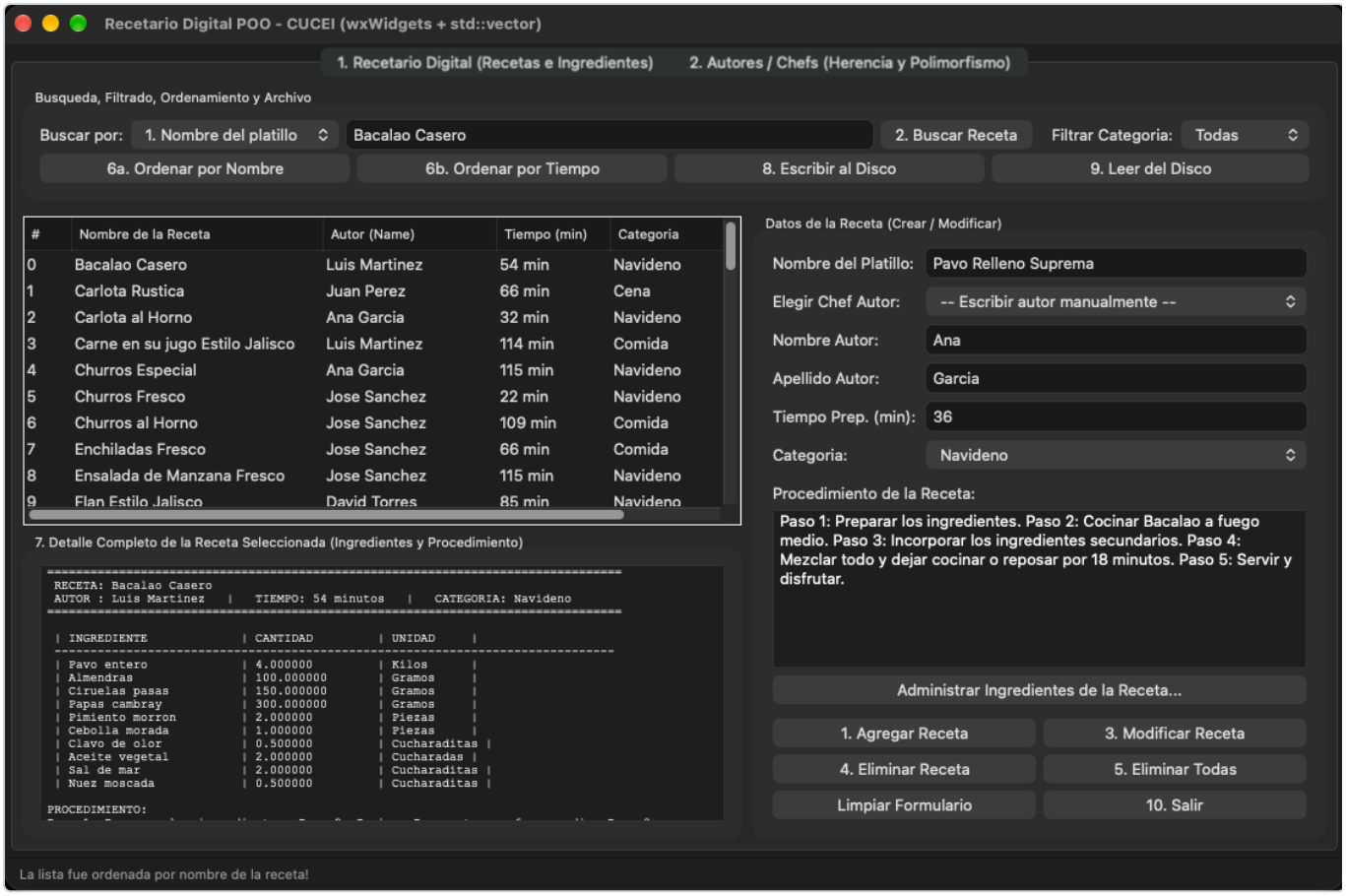



*Figura 5. Prueba de ordenamiento alfabético ascendente por nombre mediante std::sort y Recipe::compareByName sobre el std::vector<Recipe>.*

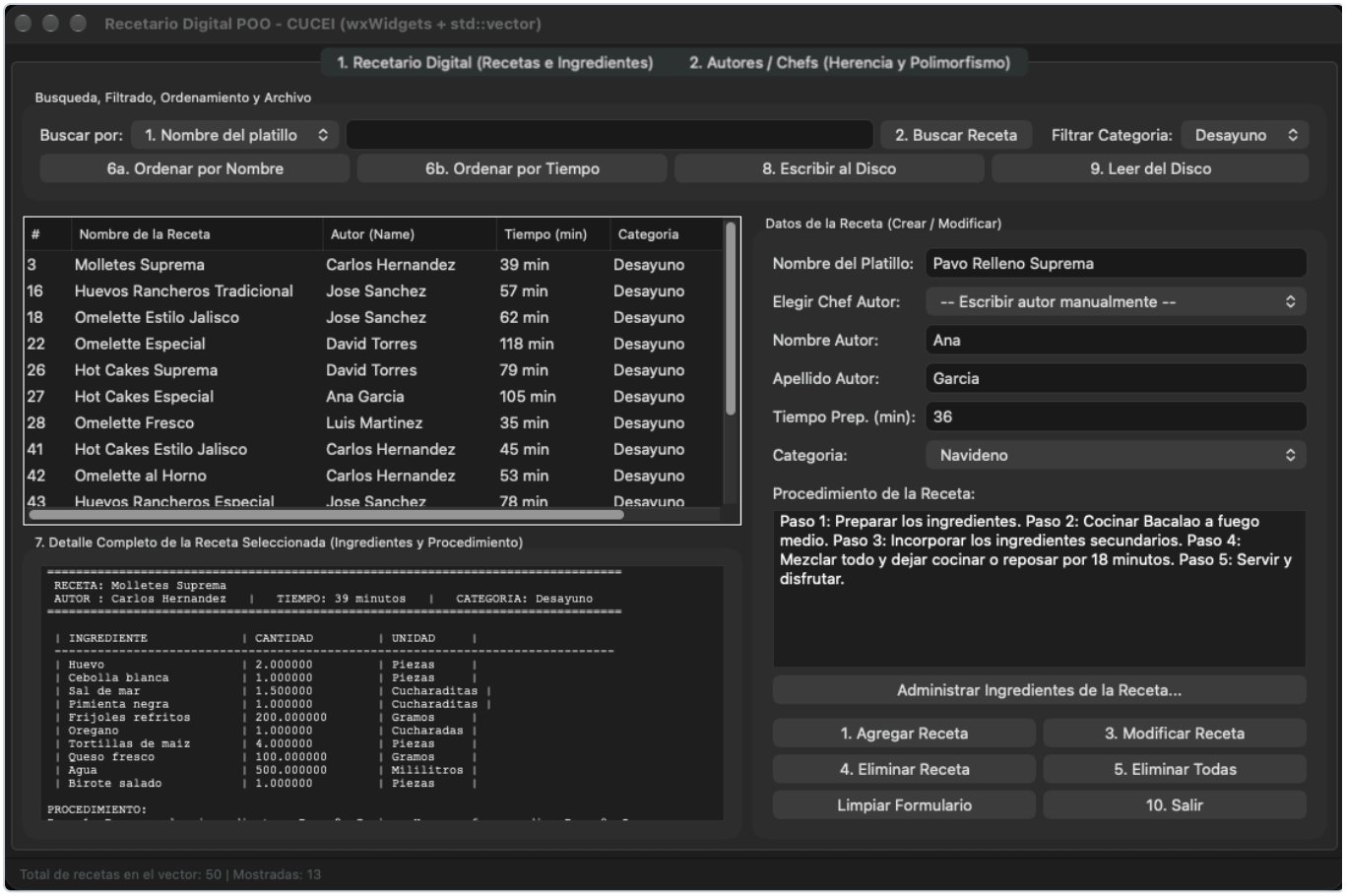



*Figura 6. Prueba de filtrado dinámico por la categoría "Desayuno", mostrando únicamente las recetas que cumplen el criterio y su ficha técnica completa a la derecha.*




*Figura 7. Ventana modal IngredientUI (hereda de wxDialog) administrando el std::vector<Ingredient> compuesto dentro de una receta (alta ordenada, edición de cantidad/unidad y eliminación).*

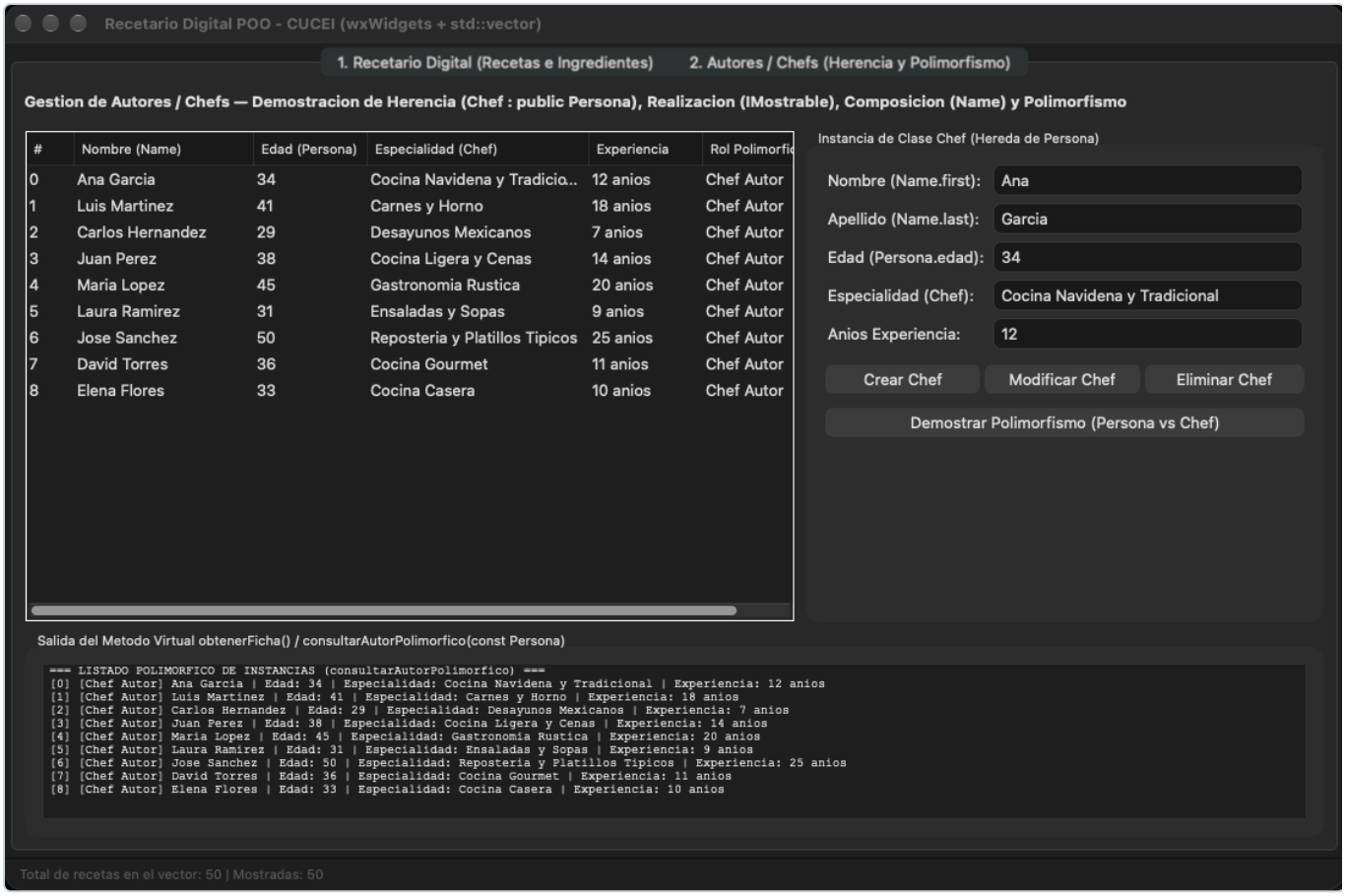



*Figura 8. Segunda pestaña de la aplicación dedicada a la jerarquía IMostrable → Persona → Chef, mostrando la ficha polimórfica obtenida por enlace dinámico (virtual / override) y la asignación directa de un Chef como autor de una receta.*

## 6. Conclusiones

El desarrollo de esta Actividad Integradora permitió consolidar de forma práctica todos los conceptos fundamentales de la **Programación Orientada a Objetos** en C++ al transformar un proyecto académico previo de consola en una aplicación gráfica completa, robusta y extensible basada en **wxWidgets**.

En primer lugar, el reemplazo de la estructura de lista estática manual por el contenedor estándar `std::vector` simplificó notablemente la gestión de memoria y el código de las clases de dominio (`Recipe`) y de interfaz (`RecipeUI` e `IngredientUI`). Al combinar `std::vector` con funciones comparadoras estáticas (`Recipe::compareByName`, `Recipe::compareByPrepTime` e `Ingredient::compareByName`) y `std::sort`, el código ganó claridad, legibilidad y seguridad sin perder ninguna de las funcionalidades originales de ordenamiento, búsqueda y persistencia en el archivo `recetas.txt`. Asimismo, durante esta migración se detectó y corrigió un detalle importante en el constructor copia de la clase `Ingredient`, comprobando cómo los contenedores de la biblioteca estándar invocan explícitamente los constructores copia de las clases al insertar o reubicar elementos en memoria.

En segundo lugar, la incorporación de la interfaz `IMostrable`, la clase base `Persona` y la clase derivada `Chef` permitió materializar los cuatro pilares de la POO dentro de un mismo caso de estudio coherente: se aplicó **abstracción** al modelar las entidades del problema, **encapsulamiento** al proteger el estado interno con validaciones en sus métodos de acceso, **herencia** tanto en el dominio gastronómico (`Chef : public Persona`) como en la arquitectura gráfica (`RecipeUI : public wxFrame` e `IngredientUI : public wxDialog`), y **polimorfismo** mediante métodos virtuales sobrescritos con `override` que se resuelven dinámicamente en tiempo de ejecución.

Finalmente, la migración de una interfaz de consola a una interfaz gráfica nativa con **wxWidgets** demostró el valor real de un diseño orientado a objetos bien desacoplado: gracias a que las clases `Name`, `Ingredient` y `Recipe` encapsulaban su propia lógica de negocio y serialización sin depender de `std::cin` o `std::cout`, fue posible construir una capa visual completamente nueva con tablas, pestañas y diálogos modales reutilizando prácticamente intacto el núcleo del modelo de datos.